

Projektbericht – Moon Carrier

Team: 3

Mitglieder: Sven Pfister, Mathias Menchini, Timo Kradolfer,
Patrick Itten, Joel Rechsteiner, Nils Kreienbühl

Studiengang: Systemtechnik

Modul: PM2

Dozent/in: Michael Wüthrich

Inhaltsverzeichnis

1	Produktentwicklung	3
1.1	Einführung und Übersicht	3
1.2	Zeitplan	4
1.3	Recherchebericht	5
1.4	Anforderungsliste	6
1.4.1	Anforderungen	6
1.4.2	Wunschanforderungen	6
1.5	Funktionsstruktur	7
1.6	Morphologischer Kasten	7
1.7	Nutzwertanalyse	9
1.7.1	Bewertungskriterien	9
1.7.2	Gewichtung	9
1.7.3	Auswertung	10
1.8	Erster Prototyp	10
1.9	Zweiter Prototyp	10
1.10	Finales Konzept	12
2	Mechanik	13
2.1	Einführung	13
2.2	Hauptkomponenten	13
2.2.1	Grundplatte	13
2.2.2	Greifarm	13
2.2.3	Antrieb	14
2.2.4	Linearachse	14
2.2.5	PES-Board und Akku	14
2.2.6	Sensoren	14
3	Elektronik	15
3.1	Elektronikschema	15
4	Software	16
4.1	Einführung und Übersicht	16
4.2	Entwicklungsumgebung	16
4.3	Softwarearchitektur	16
4.3.1	Ablauf und Aufbau	16
4.3.2	Statemachine	17
4.3.3	Line-Follower Thread	17
4.3.4	Line-Follower Regelung	17
4.3.5	Greifarm	18
5	Diskussion	19
5.0.1	Bewertung der Ergebnisse	19
5.0.2	Vergleich mit der Anforderungsliste	19
5.0.3	Erkenntnisse aus den Tests	19
5.0.4	Ausblick	19
6	Literaturverzeichnis	20
	Abbildungsverzeichnis	21
	Tabellenverzeichnis	22

A	Anhang	22
A.1	Budget	22
A.2	Pinbelegung	23
A.3	Verwendete Bauteile	23
A.4	(!!!ALT)Aufgabenstellung PDF	25
A.5	Zeitplan PDF	34
A.6	Anforderungsliste PDF	35
A.7	Funktionsstruktur PDF	38
A.8	Recherche PDF	39
A.9	main.cpp	53

1 Produktentwicklung

1.1 Einführung und Übersicht

Im Rahmen des Moduls Produktentwicklung Systemtechnik 2 (PM2) wird gemäss den VDI-Richtlinien 2221/2222 [11] ein autonomer Lieferroboter entwickelt und aufgebaut. Der Roboter trägt den Namen *Moon-Carrier*.

Die Aufgabe besteht darin, farbcodierte Pakete auf einem definierten Spielfeld autonom aufzunehmen und den entsprechend markierten Ablieferhäusern zuzuordnen. Der Roboter navigiert dabei entlang einer schwarzen Fahrlinie und muss die Pakete bei den Abholhäusern aufnehmen sowie vollständig im zugehörigen Hausfeld der Ablieferhäuser absetzen.

An die Lösung werden sowohl funktionale als auch konstruktive Anforderungen gestellt. Die maximale Startgrösse des Roboters beträgt $200\text{ mm} \times 200\text{ mm} \times 200\text{ mm}$. Zentrale technische Herausforderungen sind die zuverlässige Linienverfolgung, die Farberkennung zur Paketidentifikation, die präzise Positionierung bei den Häusern sowie eine robuste Greifmechanik für das Aufnehmen und Ablegen der Pakete.

Die Entwicklung gliedert sich in die drei mechatronischen Bereiche Mechanik, Elektronik und Software, die eng miteinander verzahnt sind. Der vorliegende Bericht dokumentiert den vollständigen Entwicklungsprozess vom Konzept bis zum Prototyp. Genauere Informationen zur Aufgabenstellung sind im Anhang unter Aufgabenstellung Challenge PM2 FS26 zu finden.

1.2 Zeitplan

Der Zeitplan wurde als Gantt-Diagramm erstellt, mit Aufgabengruppen (Planung, Entwicklung, Dokumentation) und den dazugehörigen Meilensteinen (Coaching, Roboter Version 1 etc.). Der Plan ist auf die 14 Semesterwochen mit sechs Personen ausgelegt. Aufgrund weiterer Leistungsnachweise während des Semesters sowie personeller Ausfälle im Team konnten die Vorgaben des Zeitplans nicht vollständig eingehalten werden. Der gesamte Zeitplan ist im Anhang vermerkt.

1.3 Recherchebericht

Die Vorprojektphase umfasst eine Recherche zu den drei mechatronischen Teilbereichen des MoonCarrier. In der Mechanik erweist sich ein Differentialantrieb in Kombination mit einem Servo-Greifarm als geeignetes Konzept. Die Elektronik basiert auf dem STM32 NUCLEO-F446RE sowie dem ZHAW PES Board; als Sensorik kommen ein IR-Linienarray, ein Farbsensor sowie Distanzsensoren zum Einsatz. Im Bereich Informatik übernimmt ein PD-Regler die Linienfolgeregelung, während eine Zustandsmaschine die Ablaufsteuerung strukturiert. Der vollständige Recherchebericht findet sich im Anhang.

1.4 Anforderungsliste

In der Anforderungsliste werden die wichtigsten Vorgaben für den Roboter festgehalten. Sie dient als Grundlage für die Entwicklung und zeigt, welche Funktionen, Eigenschaften und Randbedingungen erfüllt werden müssen. Dabei wird zwischen verbindlichen Anforderungen und zusätzlichen Wunsch-Anforderungen unterschieden. Siehe Anhang Anforderungsliste.

1.4.1 Anforderungen

Eine zentrale Anforderung ist, dass der Roboter Pakete mit einer Grösse von $30 \times 30 \times 30$ mm selbstständig aufnehmen, transportieren und am richtigen Ort möglichst präzise abliefern kann. Dafür ist eine zuverlässige Spurverfolgung notwendig, damit definierte Positionen sicher angefahren werden können.

Eine weitere wichtige Anforderung ist die autonome Arbeitsweise. Während des Wettbewerbs darf keine Kommunikation mit dem Roboter stattfinden. Deshalb müssen Sensorik, Messwertauswertung und Steuerung selbstständig funktionieren. Zudem muss der Roboter mit unterschiedlichen Paketpositionen und erhöhten Ablageflächen umgehen können.

Neben den funktionalen Anforderungen gibt es konstruktive Vorgaben. Die maximale Startgrösse beträgt $200 \times 200 \times 200$ mm. Falls der Tunnel genutzt wird, müssen zusätzlich dessen Grössenbegrenzungen eingehalten werden. Ausserdem sind bestimmte Komponenten wie das Nucleo-Board F446RE vorgegeben.

Auch die Wettbewerbsregeln gehören zu den Anforderungen. Der Roboter muss sich auf dem definierten Spielfeld mit Depot, Abholhäusern, Ablieferhäusern und Tunnel regelkonform bewegen. Start, Ziel, Zeitmessung und Strafzeiten sind verbindlich. Zusätzlich müssen organisatorische Vorgaben wie die termingerechte Abgabe der Projektdokumentation und des Videos eingehalten werden.

1.4.2 Wunschanforderungen

Die Wunsch-Anforderungen werden im Dokument mit W gekennzeichnet. Sie beschreiben zusätzliche Funktionen und Verbesserungen, die nicht zwingend notwendig sind, aber die Qualität des Roboters erhöhen.

Im funktionalen Bereich gehört dazu zum Beispiel die Möglichkeit, mehrere Pakete gleichzeitig zwischenzulagern und zu transportieren. Dadurch könnte der Roboter effizienter arbeiten und unnötige Fahrwege reduzieren. Ebenfalls wünschenswert ist eine intelligente Routenstrategie, mit welcher Aufnahme- und Ablieferpunkte möglichst zeitsparend angefahren werden.

Weitere Wunsch-Anforderungen betreffen den Aufbau und die Bedienbarkeit. Ein einfacher und modularer Aufbau erleichtert Wartung, Reparaturen und spätere Anpassungen. Auch ein einheitliches und ansprechendes Design trägt zu einem professionellen Gesamteindruck bei.

Im Bereich Software sind eine gute Parametrierbarkeit und sichtbare Fehlerzustände über LED-Codes wünschenswert. Dadurch wird das Testen und Optimieren vereinfacht. Zusätzlich soll der Roboter möglichst robust und sicher aufgebaut sein, zum Beispiel ohne scharfe Kanten und mit geringer Störanfälligkeit.

1.5 Funktionsstruktur

Die Funktionsstruktur (Abbildung 1) zeigt die Gesamtfunktion des Systems und deren Zerlegung in Teilfunktionen. Die Hauptaufgabe besteht darin, Pakete autonom aufzunehmen, zu transportieren, kurz zwischenzulagern und am richtigen Ort wieder abzusetzen. Dabei verarbeitet das System die Eingaben Energie, Information/Signal und Pakete und erzeugt als Ausgaben Bewegung sowie zugestellte Pakete.

Die zugeführte Energie wird zuerst gespeichert und danach zur Versorgung von Motoren, Sensoren und Elektronik genutzt. Dadurch werden alle weiteren Funktionen des Systems überhaupt erst möglich.

Parallel dazu verarbeitet das System verschiedene Informationen aus den Sensoren. Zuerst werden die Sensordaten ausgewertet, danach Linien oder Farben erkannt und daraus die aktuelle Position bzw. das Ziel bestimmt. Auf dieser Grundlage wird die Fahrbewegung gesteuert und anschliessend über Rad und Lenkung in eine reale Bewegung umgesetzt.

Für das Aufnehmen und Ablegen der Pakete sind zusätzlich Funktionen zur Erkennung der Paketposition und der Ablageposition notwendig. Diese Informationen werden benötigt, damit der Greifer gezielt bewegt werden kann.

Im unteren Bereich ist der eigentliche Paketfluss dargestellt. Die Pakete werden zuerst aufgesammelt, danach im System zwischengelagert und am Ende wieder abgesetzt. So entsteht als Endergebnis die gewünschte Zustellung der Pakete.

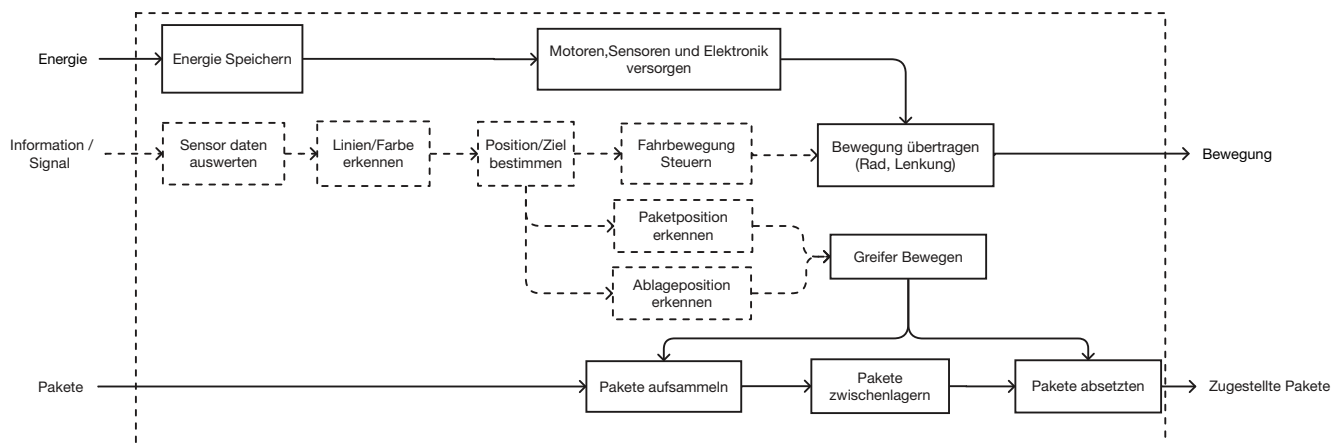


Abbildung 1: Funktionsstruktur

1.6 Morphologischer Kasten

Der morphologische Kasten strukturiert den Lösungsraum des MoonCarrier systematisch. Für jede Teilfunktion – darunter Fahrwerk, Lenkprinzip, Aufnahmemechanismus, Arm, Strategie, Paketspeicherung, Ablageprinzip sowie Routenwahl – werden mehrere Lösungsvarianten gegenübergestellt. Aus dem Gesamttraum werden drei konsistente Lösungspfade abgeleitet, die als Grundlage für die anschliessende Nutzwertanalyse dienen. Der vollständige morphologische Kasten befindet sich im Anhang.

Variante 1

Das Konzept basiert auf einem Zweiradantrieb mit Stützgleiter und Differenziallenkung. Die Paketaufnahme erfolgt elektromagnetisch über einen Gelenkarm mit zwei Freiheitsgraden. Aufgenommene Pakete werden farzunabhängig in einem Magazin zwischengespeichert und durch Abschalten des Elektromagneten abgelegt. Der Tunnel wird genutzt.

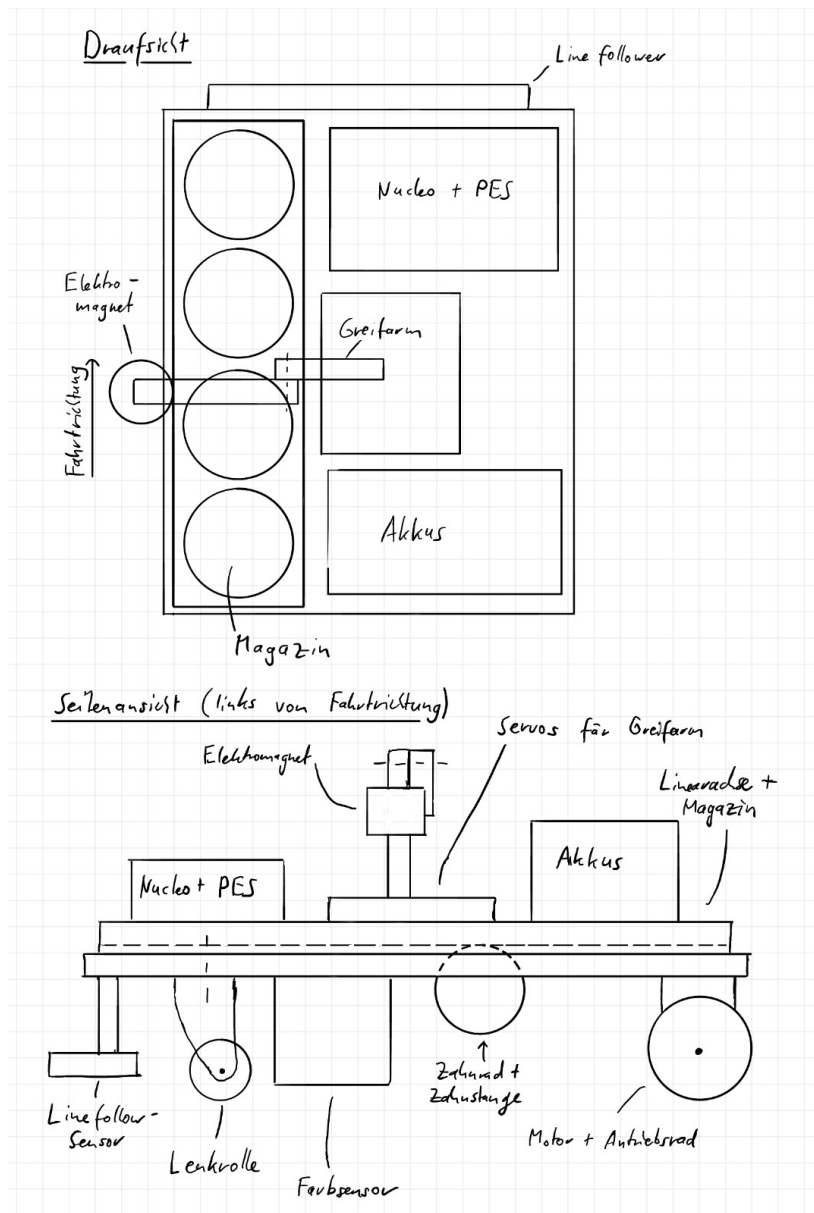


Abbildung 2: Skizze Variante 1

Variante 2

Das Konzept sieht ein Vierradfahrwerk mit Ackermann-Lenkung vor. Die Pakete werden formschlüssig über ein kartesisches Armsystem aufgenommen und direkt ohne Zwischenspeicherung zum Zielhaus transportiert. Die Ablage erfolgt kontrolliert. Der Tunnel wird in Kauf genommen.



Abbildung 3: Skizze Variante 2

Variante 3

Das Konzept verwendet ein Kettenfahrwerk mit Skid-Steering. Die Paketaufnahme erfolgt kraftschlüssig über einen Gelenkarm mit drei Freiheitsgraden. Die Pakete werden reihenfolgeabhängig auf einer offenen Ladefläche gespeichert und mittels Greifer abgelegt. Die Route wird dynamisch gewählt.



Abbildung 4: Skizze Variante 3

1.7 Nutzwertanalyse

Die drei aus dem morphologischen Kasten abgeleiteten Lösungskonzepte werden mittels einer Nutzwertanalyse systematisch bewertet und verglichen.

1.7.1 Bewertungskriterien

Die Bewertung erfolgt anhand von sechs Kriterien: Einfache Konstruktion, Zielsicherheit bzw. Ablagegenauigkeit, Betriebssicherheit, Geschwindigkeit und Strategie, Modularer Aufbau sowie Wendigkeit des Roboters.

1.7.2 Gewichtung

Die Gewichtung der Kriterien wird mittels eines Paarvergleichs ermittelt. Betriebssicherheit und Zielsicherheit erhalten dabei die höchste Gewichtung, da diese den grössten Einfluss auf die zuverlässige Aufgabenerfüllung haben.

1.7.3 Auswertung

Die gewichteten Einzelbewertungen ergeben für jede Lösung einen Gesamtnutzwert. Lösung 1 erzielt den höchsten Nutzwert und wird als Vorzugslösung für die weitere Entwicklung ausgewählt. Die vollständige Nutzwertanalyse befindet sich im Anhang.

1.8 Erster Prototyp

Abbildung 5 zeigt den ersten Prototyp. Dieser dient vor allem dazu, die grundlegende Idee des Systems praktisch umzusetzen und auszuprobieren.

Dabei liegt der Fokus klar auf der Fahrmechanik. Ziel ist es, zuerst zu prüfen, ob sich das Fahrzeug grundsätzlich bewegen lässt und ob der mechanische Aufbau mit Rädern, Antrieb und Chassis funktioniert.

Das Chassis besteht hauptsächlich aus Karton und Klebeband. Die Räder sind aus 3D Material und sind mit O-Ringe ausgestattet.

Andere Funktionen wie das gezielte Greifen oder Ablegen von Paketen stehen in dieser Phase noch nicht im Mittelpunkt.

Der Prototyp dient also hauptsächlich dazu, die fahrbare Basis zu testen und erste Erfahrungen mit dem mechanischen und elektronischen Aufbau zu sammeln.

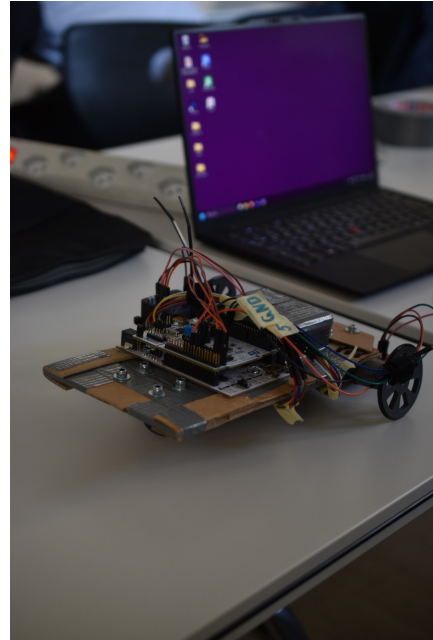


Abbildung 5: Erster Prototyp

1.9 Zweiter Prototyp

Der zweite Prototyp (Abbildung 6) baut auf den Erkenntnissen aus der ersten Version auf. Während beim ersten Aufbau vor allem die Fahrmechanik im Vordergrund steht, liegt der Fokus nun stärker auf dem Greifarm, der Optimierung der Fahrgeschwindigkeit und dem Farbsensor. Ziel ist es, das System funktional zu erweitern und gleichzeitig die bisherigen Fahrfunktionen zuverlässiger und präziser zu machen.

In dieser Version werden die aufgetretenen Schwierigkeiten deutlicher sichtbar. Während der Umsetzung müssen verschiedene Entscheidungen getroffen werden, da gewisse Funktionen mehr Zeit und Aufwand benötigen als ursprünglich erwartet.

Aus zeitlichen Gründen wird der Elektromagnet entfernt und durch einen normalen Magneten ersetzt. Der Grund dafür ist, dass der Zeitaufwand sowie die Zuverlässigkeit des Elektromagneten zu diesem Zeitpunkt nicht ausreichend abgeschätzt werden können. Durch diese Änderung funktioniert das ursprüngliche Prinzip mit dem Aufnehmen und Ablegen der Pakete nicht mehr wie geplant. Ohne Elektromagnet können die Pakete nicht gezielt auf dem Magazin abgesetzt werden.

Diese Anpassung hat auch Auswirkungen auf die Fahrstrategie. Statt mehrere Pakete in einem Durchlauf aufzunehmen, wird nun nur noch ein Paket pro Durchlauf gesammelt. Dadurch wird der Ablauf einfacher und besser kontrollierbar.

Zusätzlich wird auch die Strategie für das Rückwärtsfahren angepasst. Beim Testen zeigt sich, dass das Rückwärtsfahren schwierig umzusetzen ist und nicht zuverlässig genug funktioniert. Deshalb fährt der Roboter in dieser Version nur noch geradeaus und durch den Tunnel. Dadurch wird die Bewegung vereinfacht und die Wahrscheinlichkeit erhöht, dass der Ablauf stabil funktioniert.



Abbildung 6: Zweiter Prototyp

1.10 Finales Konzept

Die Nutzwertanalyse hat Variante 1 als Vorzugslösung identifiziert. Das ursprüngliche Konzept sah vor, alle Pakete nacheinander mittels Elektromagnet auf ein Magazin zu laden, anschliessend einmalig durch den Tunnel zu fahren und die Pakete auf der gegenüberliegenden Seite einzeln abzulegen. Der wesentliche Vorteil dieser Strategie bestand darin, den Parcours nur einmal zu durchfahren, was eine erhebliche Zeitersparnis bedeutet hätte.

Im Verlauf des Projekts traten personelle Ausfälle im Team auf, wodurch die ursprünglich geplante Aufgabenverteilung nicht mehr aufrechterhalten werden konnte. Die Entwicklung des Elektromagneten sowie der dazugehörigen Aufnahme- und Lagerlogik in der Software blieb dadurch unrealisiert.

Aufgrund des entstehenden Zeitdrucks wurde eine Vereinfachung des Konzepts vorgenommen: Der Greifarm wird direkt auf dem Magazin befestigt und nimmt jeweils nur ein Paket pro Durchlauf auf. Statt einer einmaligen Fahrt wird der Parcours mehrfach durchfahren. Durch diesen Ansatz entfällt der Elektromagnet vollständig, und die komplexe Magazin- und Lagerlogik in der Software wird eliminiert.

Diese Entscheidung erwies sich jedoch als unzureichende Vereinfachung: Der Greifarm allein stellte bereits eine erhebliche mechanische und softwareseitige Herausforderung dar und war massgeblich daran beteiligt, dass die verfügbare Entwicklungszeit überschritten wurde. Die Abweichungen vom ursprünglichen Konzept sind in Tabelle 1 zusammengefasst.

Entsprechend wird auch im folgenden Kapitel (Mechanik 2) das Finale Konzept und nicht der Lösungsvorschlag 1 behandelt.

Tabelle 1: Abweichungen vom Konzept der Nutzwertanalyse

Merkmal	NWA Variante 1	Finales Produkt	Begründung
Aufnahme	Elektromagnet	Permanentmagnet	Elektromagnet wurde aufgrund personeller Ausfälle nicht realisiert
Speicherung	Magazin (mehrere Pakete)	Keine (1 Paket pro Durchlauf)	Entfällt ohne Elektromagnet; vereinfacht die Steuerungslogik
Fahrstrategie	Einmalige Rundfahrt	Mehrfache Rundfahrten	Konsequenz aus dem Wegfall der Magazinspeicherung
Arm	Gelenkarm 2 DOF	Gelenkarm + Linearachse	Präzisere Positionierung beim Aufnehmen der Pakete erforderlich



Abbildung 7: Finales Konzept

2 Mechanik

2.1 Einführung

Der Aufbau des Roboters ist relativ simpel gehalten, die meisten Komponenten sind verschraubt. So können mechanische Anpassungen schnell durchgeführt und getestet werden. In Abbildung 8 ist das CAD-Modell des zusammengebauten Roboters ersichtlich, die Fahrtrichtung ist nach schräg-links-unten.

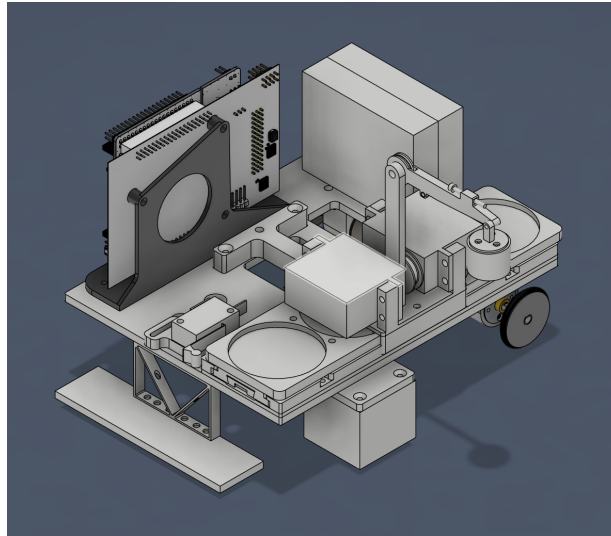


Abbildung 8: CAD-Modell des Roboters

2.2 Hauptkomponenten

2.2.1 Grundplatte

Die Grundplatte dient als Basis, auf welcher alle anderen Komponenten montiert werden (Abb. 9). Die Dicke der Grundplatte beträgt 4,2 mm und sie ist 3D-gedruckt. Somit ist es möglich, Gewindeeinsätze einzuschmelzen, an welchen die restlichen Komponenten angeschraubt werden können.

2.2.2 Greifarm

Der Greifarm (Abb. 10) dient dazu, die Pakete aufzuheben und an der Lieferadresse wieder abzulegen. Er wird über zwei Servomotoren bewegt und verfügt über einen Neodym-Magneten, um die Pakete anzuheben. Der untere Arm wird direkt von einem Servomotor bewegt. Der obere Arm wird über ein Seil bewegt, welches um eine Trommel gewickelt ist. Beim Bewegen des unteren Armes wird das Seil auf der einen Seite aufgewickelt und auf der anderen Seite abgewickelt. Somit ist die Position des oberen Armes nicht abhängig von der Position des unteren Armes.

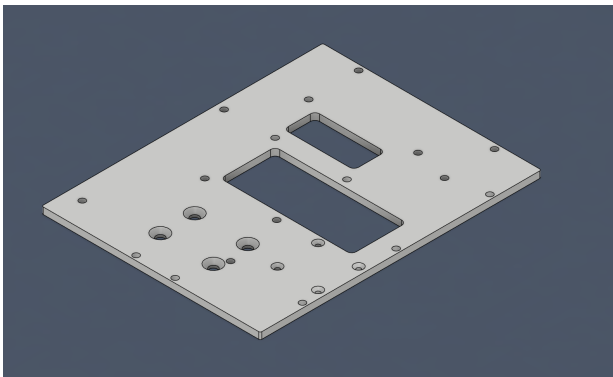


Abbildung 9: Grundplatte

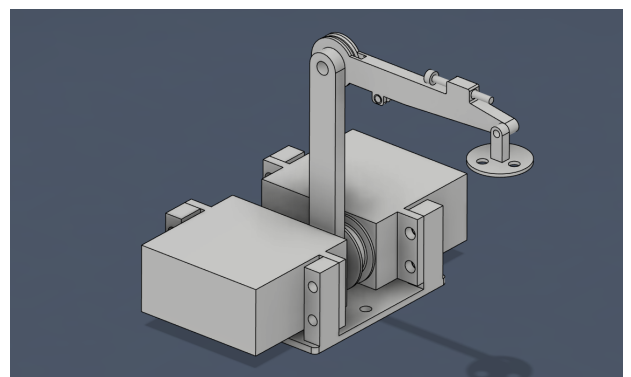


Abbildung 10: Greifarm (ohne Seil)

2.2.3 Antrieb

Der Antrieb wird über zwei DC-Servomotoren am Heck des Fahrzeugs nach dem Differentialantriebsprinzip [12] umgesetzt. Um beispielsweise nach rechts zu lenken, muss der linke Motor schneller drehen. Damit die Front des Roboters gestützt ist, wurde eine Lenkrolle montiert, welche sich 360° drehen kann (Abb. 11).

2.2.4 Linearachse

Der Greifarm ist auf einer Linearachse montiert (Abb. 12). Dies ermöglicht es, an der Haltelinie zu stoppen und anschliessend den Arm in die richtige Position zu fahren, um das Paket zu greifen. Die Linearachse ist durch eine simple Kastenführung geführt und wird über eine Zahnstange an der Unterseite des Magazins angetrieben. Die Zahnstange wird durch einen DC-Motor unter der Grundplatte angetrieben und kann über einen Mikroschalter am vorderen Teil des Roboters referenziert werden (Abb. 11).

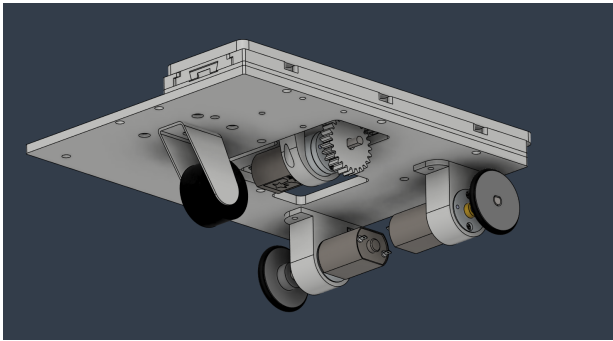


Abbildung 11: Unterseite mit Rädern und Antrieb für Linearführung

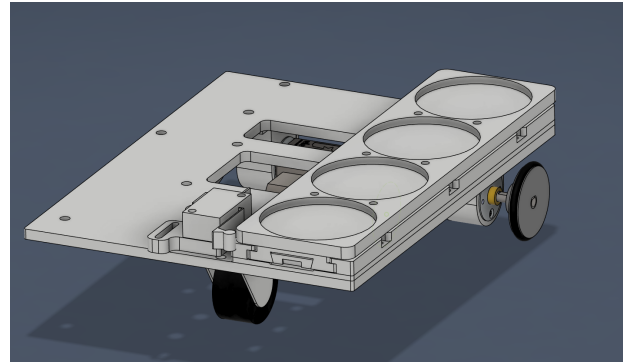


Abbildung 12: Linearführung (ohne Greifarm)

2.2.5 PES-Board und Akku

Das PES-Board ist in Fahrtrichtung rechts vorne über eine Adapterplatte auf der Grundplatte montiert. So befindet es sich relativ nah an allen Sensoren und Aktoren und ausserhalb des Weges der Mechanik. Das PES-Board steht senkrecht auf der Grundplatte, um Platz zu sparen. Der Akku ist gleich hinter dem PES-Board montiert, da die Akkukabel sehr kurz sind (Abb. 13).

2.2.6 Sensoren

Ganz vorne am Roboter ist der Linefollow-Sensor montiert (Abb. 14). Auf der linken Seite des Roboters ist der Farbsensor montiert. Er steht leicht über die Seite des Roboters hinaus, sodass er genau auf dem Farbfeld zu stehen kommt.

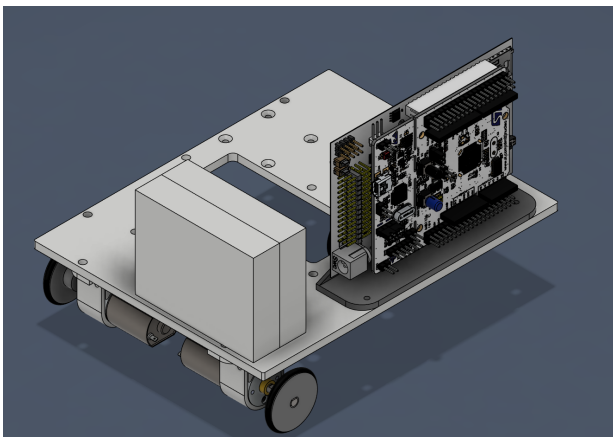


Abbildung 13: PES-Board und Akku

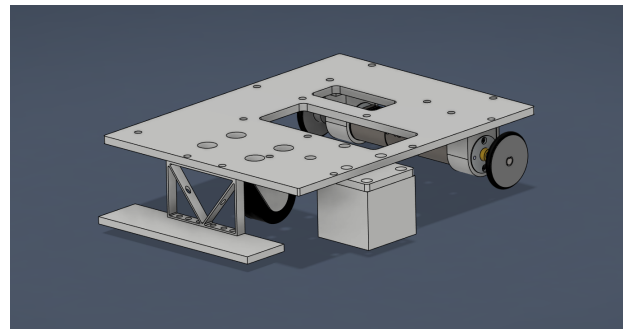


Abbildung 14: Linefollow- und Farbsensor

3 Elektronik

Die Elektronik bildet die Schnittstelle zwischen Mechanik und Software und übernimmt sowohl die Verarbeitung der Sensordaten als auch die Ansteuerung der Aktoren.

Das Gesamtsystem besteht aus zwei Hauptkomponenten: einem Mikrocontroller-Board (Nucleo-Board F446RE [10]) sowie einem speziell entwickelten Motherboard der ZHAW [14].

Das Nucleo-Board basiert auf einem ARM-Mikrocontroller und stellt die zentrale Recheneinheit des Systems dar. Es übernimmt die Verarbeitung der Sensordaten, die Regelalgorithmen zur Linienverfolgung sowie die Steuerung der Motoren. Die Programmierung erfolgt in C++ über die Mbed-Plattform.

3.1 Elektronikschema

In Abbildung 15 ist das vereinfachte Elektronikschema dargestellt. Die Kernkomponente der Schaltung bilden das Nucleo-Board und das PES-Board, über welche sämtliche Komponenten angesteuert und ausgelesen werden. Auch die Spannungsversorgung der elektronischen Bauteile erfolgt über diese beiden Boards.

Zu den Aktoren gehören drei DC-Motoren sowie zwei Servomotoren. Die Motoren M1 und M2 sind für den Antrieb der Räder zuständig und ermöglichen die Fortbewegung des MoonCarriers. Der Motor M3 bewegt das Lager, auf dem sich der Greifarm befindet. Die beiden Servomotoren werden für die Bewegungen des Greifmechanismus eingesetzt.

Die für den autonomen Betrieb benötigten Informationen erhält das System über drei Sensoren. Der Line-Follower dient zur Erkennung und Verfolgung der schwarzen Linie. Der Farbsensor erkennt die farbigen Markierfelder der Häuser, während der User-Button zum Starten des MoonCarriers verwendet wird.

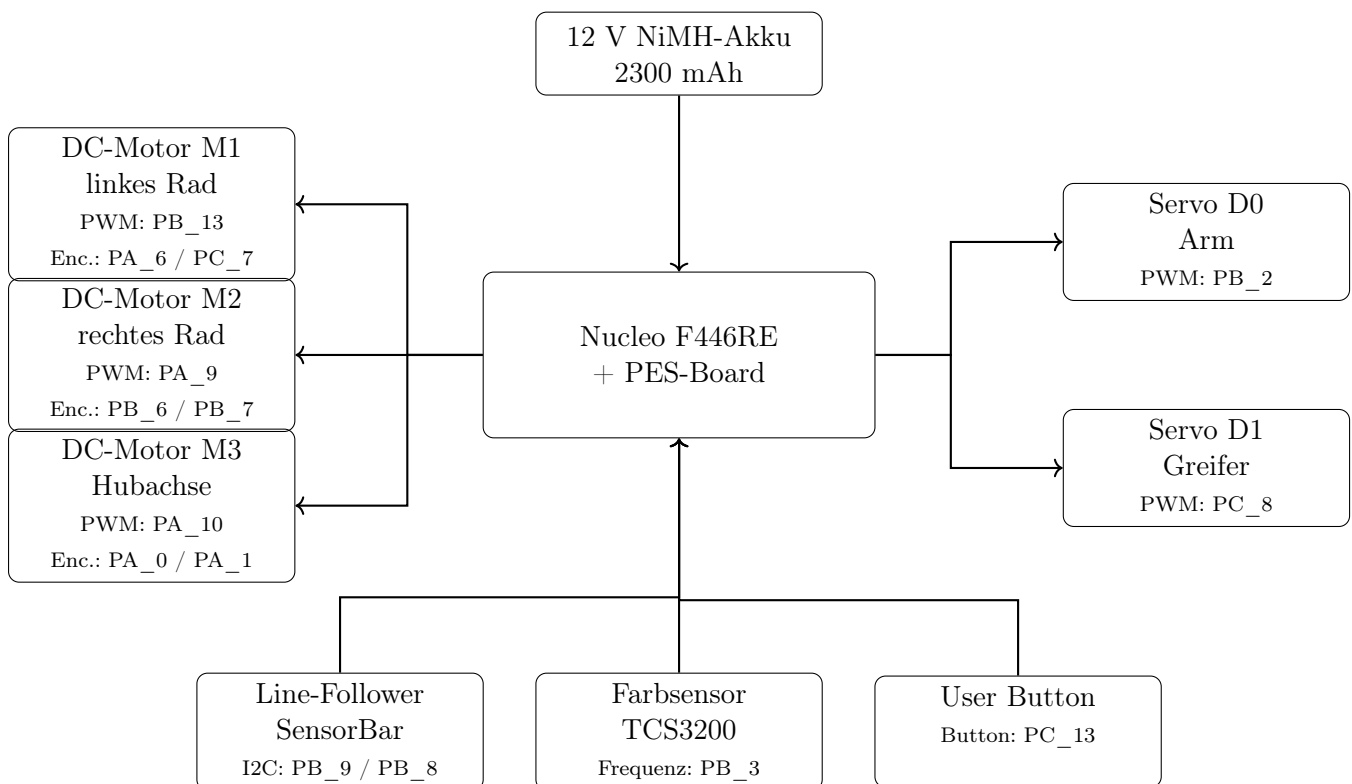


Abbildung 15: Vereinfachtes Elektrodiagramm des Roboters

4 Software

4.1 Einführung und Übersicht

Das Programm steuert einen differenzialgetriebenen Roboter auf dem STM32 Nucleo F446RE unter Mbed OS. Die Ablaufsteuerung ist als Zustandsmaschine implementiert, die zwischen Linienverfolgung, Farberkennung und Kransteuerung wechselt. Ein separater Hintergrund-Thread überwacht parallel dazu die Kreuzungserkennung. Der blaue Button startet und stoppt den gesamten Betrieb.

4.2 Entwicklungsumgebung

Der Roboter wird in C++ programmiert, einer weit verbreiteten Sprache für hardwarenahe Systeme. Als Betriebssystem kommt Mbed OS [1] zum Einsatz, das grundlegende Funktionen wie Zeitmessung, Threads und Hardwarezugriff bereitstellt. Gebaut wird das Projekt mit PlatformIO [6], einem modernen Build-System, das Kompilierung und das Übertragen der Firmware auf den Mikrocontroller vereinfacht.

Für spezifische Hardwarekomponenten – wie Motoren, Sensoren und Servos – werden eigene Bibliotheken verwendet, die im Rahmen des Kurses bereitgestellt wurden. Mathematische Berechnungen, etwa für die Fahrkinematik, nutzen die Bibliothek Eigen [5].

Der gesamte Quellcode wird mit Git [8] versioniert, wodurch Änderungen nachvollziehbar dokumentiert und frühere Versionen bei Bedarf wiederhergestellt werden können.

4.3 Softwarearchitektur

4.3.1 Ablauf und Aufbau

Das Programm hat zwei Hauptkomponenten: '(1) Main-Thread' und der '(2) Line-Follow-Thread'. Ein "Thread" kann man sich hier wie einen eigenen Prozess vorstellen. Das Programm besitzt also zwei voneinander getrennte Prozesse. Die Kommunikation zwischen den Threads findet über das beschreiben von globalen Variablen statt.

Der Main-Thread ist eine Schleife die eine Durchlaufzeit von 20ms hat, also eine Frequenz von 50Hz. Der Main-Thread behandelt die Grundsätzliche Logik des Roboters: Initialisieren, Fahren, Stoppen, Farben erkennen, Packetbehandlung. Die Arbarbeitung dieser Zustände bzw. States wird mittels dem State-Machine-Pattern gelöst. Auf die genaue Eigenschaften der einzelnen States, wird im Kapitel 4.3.2 (Statemachine) eingegangen.

Der Line-Follow-Thread läuft parallel zur Hauptschleife und ist ausschliesslich dafür zuständig, den Li-niensensor auszulesen und Markierungslinien auf dem Boden zu erkennen. Er arbeitet mit einer Abtastrate von 250Hz – also 250 Messungen pro Sekunde – und ist damit fünfmal schneller als die Hauptschleife des Roboters.

Diese höhere Frequenz wurde bewusst gewählt: Bei zu geringer Abtastrate würde der Roboter bei höheren Fahrgeschwindigkeiten über eine Markierungslinie hinwegfahren, ohne sie zu erkennen. Durch den schnelleren Thread kann die Fahrgeschwindigkeit erhöht werden, ohne dabei die zuverlässige Erkennung der Markierungen zu gefährden. Wird eine Linie erkannt, setzt der Thread ein Flag, das die Hauptschleife beim nächsten Zyklus auswertet und den Roboter entsprechend reagieren lässt.

Das STM32 Nucleo F446RE verfügt über zwei User-Buttons. Wird der blaue gedrückt startet das Programm mit der Main-Loop und der Roboter fängt an zu fahren. Wird der schwarze gedrückt wird der Roboter zurückgesetzt und ist am warten.

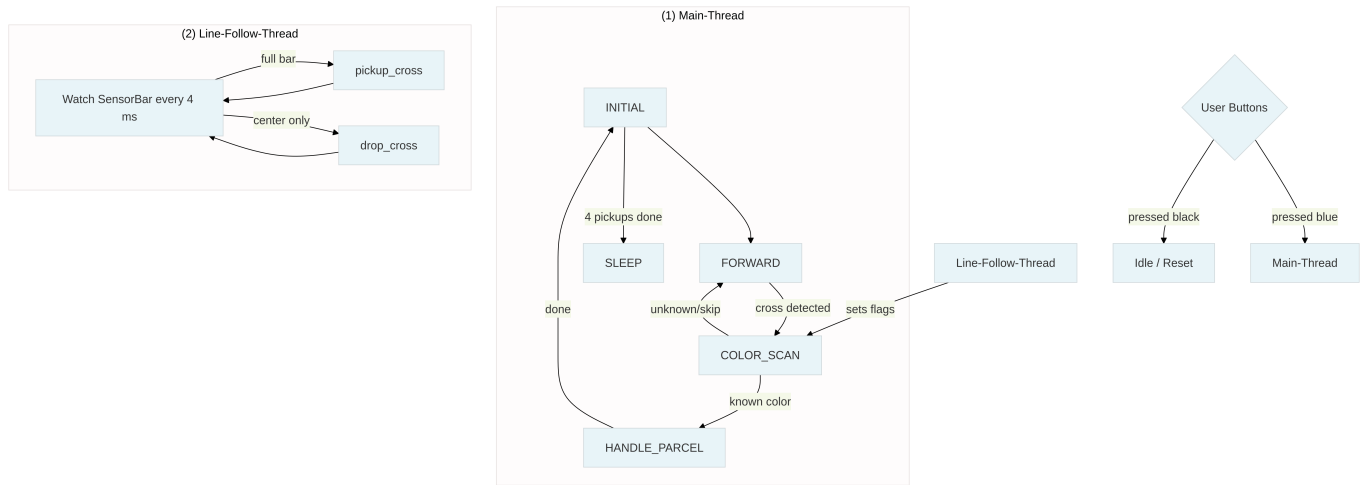


Abbildung 16: Ablaufdiagramm mit (1) Main-Thread, (2) Line-Follow-Thread und User-Button

4.3.2 Statemachine

Die Zustandsmaschine (Finite State Machine [13]) strukturiert die Ablaufsteuerung. Sie definiert, in welchem Zustand sich der Roboter gerade befindet und welche Aktionen er ausführt. Zu jedem Zeitpunkt ist genau ein Zustand aktiv. Ein Zustandswechsel erfolgt nur, wenn eine bestimmte Bedingung erfüllt ist – etwa wenn eine Markierungslinie erkannt oder eine Farbe identifiziert wurde. Die folgende Tabelle gibt einen Überblick über alle Zustände und ihre Funktion.

Zustand	Beschreibung
INITIAL	Servos fahren in Ausgangsposition, Flags werden zurückgesetzt
FORWARD	Roboter folgt der Linie
COLOR_SCAN	Roboter stoppt und identifiziert die Farbe der Markierung
HANDLE_PARCEL	Greifarm nimmt ein Paket auf oder legt es ab
SLEEP	Alle Aufgaben erledigt, Roboter bleibt stehen

Tabelle 2: Übersicht der Zustände der Statemachine

Zu erwähnen ist, dass der Zustand `HANDLE_PARCEL` selbst wiederum als eigene Statemachine implementiert ist, die jeden einzelnen Schritt der Armbewegung als separaten Zustand definiert. Im Rahmen dieses Berichts wird darauf nicht weiter eingegangen – eine detaillierte Beschreibung findet sich in Anhang ??.

4.3.3 Line-Follower Thread

Der Line-Follower-Thread liest den Liniensensor kontinuierlich aus und analysiert dessen Muster. Anhand des Musters wird unterschieden, ob der Roboter eine Abholmarkierung, eine Ablagemarkierung oder keine relevante Linie überquert. Sobald eine Markierung erkannt wird, setzt der Thread ein Flag, das die Hauptschleife beim nächsten Zyklus auswertet. Um eine Race-Condition zu vermeiden wird der Line-Follow-Thread nach dem Aufnehmen und Ablegen von Paketen für 0,5 sec pausiert. Technische Details zur Implementierung sind in Anhang ?? dokumentiert.

4.3.4 Line-Follower Regelung

Damit der Roboter der Linie möglichst präzise folgt, kommt ein PD-Regler zum Einsatz. Dieser berechnet anhand der aktuellen Position der Linie relativ zum Roboter, wie stark gegengesteuert werden muss. Weicht der Roboter nach links ab, dreht der Regler nach rechts – und umgekehrt. Das Ergebnis ist eine flüssige, stabile Linienverfolgung auch bei höheren Geschwindigkeiten.

4.3.5 Greifarm

Der Greifarm übernimmt das Aufnehmen und Ablegen der Pakete. Die Bewegungssequenz ist in fünf Schritte unterteilt: Zuerst öffnet sich der Greifer, dann fährt der Arm aus, anschliessend bewegt sich der Kran zur Zielposition, fährt zurück und kehrt schliesslich in die Ausgangsposition zurück. Die genaue Zielposition des Krans hängt dabei von der erkannten Farbe des Pakets ab, da jede Farbe einem anderen Ablageort entspricht. Die Werte für die jeweilige Zielposition werden empirisch bestimmt und sind hart kodiert.

5 Diskussion

In diesem Kapitel werden die erzielten Ergebnisse des Projekts diskutiert und hinsichtlich ihrer Erwartbarkeit, Aussagekraft und Relevanz beurteilt. Dabei wird betrachtet, inwiefern die umgesetzte Lösung die Anforderungen erfüllt und welche Erkenntnisse aus Entwicklung, Tests und Wettbewerb gewonnen werden konnten. Zusätzlich werden mögliche Verbesserungen sowie Ansätze für eine Weiterentwicklung des Roboters aufgezeigt.

5.0.1 Bewertung der Ergebnisse

In diesem Abschnitt werden die erreichten Ergebnisse beurteilt. Dabei wird beschrieben, welche Funktionen des Roboters zuverlässig umgesetzt werden konnten und bei welchen Funktionen Abweichungen oder Probleme aufgetreten sind.

5.0.2 Vergleich mit der Anforderungsliste

Die wichtigsten Muss-Anforderungen wurden grösstenteils erfüllt. Der Roboter kann sich autonom bewegen, der Linie folgen und die Pakete aufnehmen sowie wieder ablegen. Auch die vorgegebenen Bauteile wie Nucleo-Board, PES-Board, Motoren, Servos, Line-Follower und Farbsensor wurden eingesetzt. Die maximale Startgrösse wurde bei der Konstruktion ebenfalls berücksichtigt.

Nicht alle Wunsch-Anforderungen konnten vollständig erreicht werden. Besonders die Zwischenlagerung mehrerer Pakete auf dem Roboter wurde nicht umgesetzt. Auch die intelligente Routenstrategie konnte nur eingeschränkt realisiert werden, da der Fokus stärker auf der Grundfunktion lag. Zusätzlich besteht bei der Wiederholbarkeit und bei der genauen Positionierung der Pakete noch Verbesserungspotenzial.

Zusammenfassend erfüllt der Roboter die wichtigsten Grundanforderungen. Die Wunsch-Anforderungen wurden jedoch nur teilweise erreicht und bieten Raum für weitere Verbesserungen.

5.0.3 Erkenntnisse aus den Tests

Bei der Feinjustierung sollte immer nur eine Änderung auf einmal vorgenommen werden. Dadurch bleibt besser nachvollziehbar, welche Anpassung welchen Einfluss auf das Verhalten des Roboters hat. Dies erleichtert die Fehlersuche und verhindert, dass mehrere Änderungen gleichzeitig neue Probleme verursachen. Beim Feinjustieren ist es sinnvoll, zuerst den Parameter mit dem grössten Einfluss anzupassen und danach schrittweise die weniger wichtigen Parameter zu bearbeiten.

Wenn mechanische Änderungen am Roboter vorgenommen werden, sollten die bisherigen Parameter nicht einfach übernommen werden. Stattdessen ist es sinnvoll, die relevanten Parameter neu abzustimmen und bei Bedarf wieder von einem Ausgangswert aus zu beginnen. So kann verhindert werden, dass alte Einstellungen nicht mehr zur veränderten Mechanik passen.

Für zukünftige Tests wäre es hilfreich, Mess- und Fahrdaten auf einer SD-Karte zu speichern. Dadurch könnte nach einem Testlauf genauer analysiert werden, wie sich der Roboter verhalten hat und an welchen Stellen Abweichungen entstanden sind.

Funktionierende Einstellungen sollten möglichst beibehalten werden. Änderungen sollten nur dann vorgenommen werden, wenn sie wirklich notwendig sind oder ein klares Problem beheben.

5.0.4 Ausblick

Im Ausblick wird beschrieben, wie der Roboter in einer weiteren Entwicklungsphase verbessert oder erweitert werden könnte. Dabei kann auch kurz auf die Machbarkeit dieser Weiterentwicklungen eingegangen werden.

6 Literaturverzeichnis

- [1] Arm Limited. *Mbed OS Documentation*. Abgerufen am 06.05.2025. 2024. URL: <https://os.mbed.com/docs/mbed-os/>.
- [2] BerryBase. *TCS230/TCS3200 Farbsensor-Modul*. Abgerufen am 06.05.2025. 2024. URL: <https://www.berrybase.ch/tcs230-tcs3200-farbsensor-modul>.
- [3] Conrad Electronic. *Reely Modellbau-Akkupack NiMH 6V 2300mAh*. Abgerufen am 06.05.2025. 2024. URL: <https://www.conrad.ch/de/p/reely-modellbau-akkupack-nimh-6-v-2300-mah-zellen-zahl-5-mignon-aa-side-by-side-jr-buchse-2613252.html>.
- [4] Conrad Electronic. *Reely Standard-Servo CYS-S0090*. Abgerufen am 06.05.2025. 2024. URL: <https://www.conrad.ch/de/p/reely-standard-servo-cys-s0090-analog-servo-getriebe-material-metall-stecksystm-jr-2203091.html>.
- [5] Gaël Guennebaud, Benoît Jacob u. a. *Eigen – C++ Template Library for Linear Algebra*. Abgerufen am 06.05.2025. 2024. URL: <https://eigen.tuxfamily.org>.
- [6] PlatformIO Labs. *PlatformIO: A Professional Collaborative Platform for Embedded Development*. Abgerufen am 06.05.2025. 2024. URL: <https://platformio.org>.
- [7] Pololu Corporation. *20D Metal Gearmotors*. Abgerufen am 06.05.2025. 2024. URL: <https://www.pololu.com/category/167/20d-metal-gearmotors>.
- [8] Software Freedom Conservancy. *Git – Distributed Version Control System*. Abgerufen am 06.05.2025. 2024. URL: <https://git-scm.com>.
- [9] SparkFun Electronics. *SparkFun Line Follower Array*. Abgerufen am 06.05.2025. 2024. URL: <https://www.sparkfun.com/sparkfun-line-follower-array.html>.
- [10] STMicroelectronics. *NUCLEO-F446RE Product Page*. Abgerufen am 06.05.2025. 2024. URL: <https://www.st.com/en/evaluation-tools/nucleo-f446re.html>.
- [11] Verein Deutscher Ingenieure. *VDI 2221 – Methodik zum Entwickeln und Konstruieren technischer Systeme und Produkte*. Düsseldorf, 2019.
- [12] Wikipedia. *Differential wheeled robot*. Abgerufen am 06.05.2025. 2025. URL: https://en.wikipedia.org/wiki/Differential_wheeled_robot.
- [13] Wikipedia. *Endlicher Automat*. Abgerufen am 06.05.2025. 2025. URL: https://de.wikipedia.org/wiki/Endlicher_Automat.
- [14] ZHAW PM2 ADHS. *PES Board – Platform for Embedded Systems*. Abgerufen am 06.05.2025. 2024. URL: <https://github.com/ZHAW-PM-2-ADHS/ADHS>.

Abbildungsverzeichnis

1	Funktionsstruktur	7
2	Skizze Variante 1	8
3	Skizze Variante 2	9
4	Skizze Variante 3	9
5	Erster Prototyp	10
6	Zweiter Prototyp	11
7	Finales Konzept	12
8	CAD-Modell des Roboters	13
9	Grundplatte	13
10	Greifarm (ohne Seil)	13
11	Unterseite mit Rädern und Antrieb für Linearführung	14
12	Linearführung (ohne Greifarm)	14
13	PES-Board und Akku	14
14	Linefollow- und Farbsensor	14
15	Vereinfachtes Elektrodiagramm des Roboters	15
16	Ablaufdiagramm mit (1) Main-Thread, (2) Line-Follow-Thread und User-Button	17

Tabellenverzeichnis

1	Abweichungen vom Konzept der Nutzwertanalyse	12
2	Übersicht der Zustände der Statemachine	17
3	Budgetliste der verwendeten Bauteile	22

Deklaration des Einsatzes von generativer KI

Im Rahmen dieser Arbeit wurden generative KI-Werkzeuge in verschiedenen Prozessphasen unterstützend eingesetzt. Dabei wurden folgende Tools verwendet:

- **ChatGPT**: zur Verbesserung von Formulierungen, zur sprachlichen Überarbeitung sowie zum Verständnis einzelner Inhalte und Zusammenhänge.
- **claude code**: wurde zur Unterstützung beim Programmieren verwendet.
- **claude.ai**: zur Verbesserung von Formulierungen, zur sprachlichen Überarbeitung sowie zum Verständnis einzelner Inhalte und Zusammenhänge.
- **[Weiteres Tool]**: für **[Zweck einfügen]**.

Die inhaltliche Verantwortung für diese Arbeit liegt vollständig bei den Autor:innen. Alle durch KI erzeugten oder unterstützten Inhalte wurden kritisch geprüft, hinsichtlich Relevanz und Wahrheitsgehalt überprüft und bei Bedarf überarbeitet. Verwendete Quellen wurden eigenständig kontrolliert und, wo erforderlich, korrekt referenziert.

Falls in der Arbeit Artefakte wie Texte, Bilder, Code oder andere Inhalte aus bestehenden Quellen verwendet oder mit KI bearbeitet wurden, wurden die entsprechenden Originalquellen angegeben.

A Anhang

A.1 Budget

Für zusätzliche Hardware steht ein Budget von CHF 75.– zur Verfügung. Wenn möglich werden Komponenten aus dem Fundus verwendet. Diese werden virtuell mit 50 % ihres Wertes dem Budget belastet.

Tabelle 3: Budgetliste der verwendeten Bauteile

Anzahl	Bauteilname	Funktion	Verwendung	Kosten
1	DC-Motor	Antrieb des Roboters	Die Motoren bewegen den Roboter und ermöglichen die Fahrt durch den Parcours.	CHF 0.–
1	Magnet	Aufnehmen der Pakete	Der Magnet wird verwendet, um ein Paket aufzunehmen und während der Fahrt zu halten.	CHF 0.–

A.2 Pinbelegung

Kategorie	Komponente	Signal	PES-Symbol	STM32-Pin	Bemerkung
DC Motor	Motor M1 (linkes Rad)	PWM	PB_PWM_M1	PB_13	linkes Rad
DC Motor	Motor M1	Encoder A	PB_ENC_A_M1	PA_6	linkes Rad
DC Motor	Motor M1	Encoder B	PB_ENC_B_M1	PC_7	linkes Rad
DC Motor	Motor M2 (rechtes Rad)	PWM	PB_PWM_M2	PA_9	rechtes Rad
DC Motor	Motor M2	Encoder A	PB_ENC_A_M2	PB_6	rechtes Rad
DC Motor	Motor M2	Encoder B	PB_ENC_B_M2	PB_7	rechtes Rad
DC Motor	Motor M3 (Hubachse)	PWM	PB_PWM_M3	PA_10	Hubachse
DC Motor	Motor M3	Encoder A	PB_ENC_A_M3	PA_0	Hubachse
DC Motor	Motor M3	Encoder B	PB_ENC_B_M3	PA_1	Hubachse
DC Motor	Alle Motoren	Enable	PB_ENABLE_ DC-MOTORS	PB_15	Um die Motoren zu aktivieren
Servo	Servo D0	PWM	PB_D0	PB_2	Arm
Servo	Servo D1	PWM	PB_D1	PC_8	Greifer
Sensor	SensorBar (Linienarray)	I2C SDA		PB_9	Im Konstruktor festgelegt
Sensor	SensorBar (Linienarray)	I2C SCL		PB_8	Im Konstruktor festgelegt
Sensor	ColorSensor (TCS3200)	Frequenzausgang		PB_3	Single-Pin-Konstruktor
Board	User Button	Digital In	BUTTON1	PC_13	Blauer Taster auf dem Nucleo
Board	User LED	Digital Out	LED1	PA_5	Grüne LED auf dem Nucleo

A.3 Verwendete Bauteile

Anzahl	Aktoren	Verwendung	Internetlink
3	DC-Motoren mit Encoder	Antrieb der Räder sowie Positionierung der Achse des Roboterarms.	pololu [7]
2	RC-Servos	Gezielte Positionierung des Roboterarms zur Aufnahme und Ablage der Pakete	Conrad [4]

Anzahl	Sensoren	Verwendung	Internetlink
1	Line-Follower-Array	Kontinuierliche Erfassung der Fahrbahnlinie zur Regelung der Fahrtrichtung	sparkfun [9]
1	Farbsensor	Erfassung der Farbcodierung auf dem Spielfeld	berrybase [2]

Anzahl	Mikrocontroller	Verwendung	Internetlink
1	ZHAW-Motherboard PES-Board	Das PES-Board erweitert das Nucleo F446RE um zusätzliche Schnittstellen, Sensorik und Leistungselektronik	github [14]
1	Nucleo-Board F446RE	Ausführung der Software zur Verarbeitung der Sensorwerte und Ansteuerung der Motoren und Servos	ST [10]

Anzahl	Spannungsversorgung	Verwendung	Internetlink
1	Akkupaket: 12 V, 2300 mAh	Spannungsversorgung	conrad [3]

Als Energiequelle dient ein Akkupaket bestehend aus zwei in Serie geschalteten NiMH-Akkus mit jeweils fünf Zellen. Daraus resultiert eine Nennspannung von 12 V bei einer Kapazität von 2300 mAh. Das Akkupaket stellt die Versorgung für die Elektronik sowie die Antriebskomponenten des Systems sicher.

Aufgabenstellung Challenge PM2 FS26

Ihr Team hat von der Firma «LineXpress – we deliver along the line» den Auftrag erhalten, einen Roboter zu entwickeln und bauen, welcher in der Lage ist, Pakete bei Häusern abzuholen und bei anderen Häusern abzuliefern. Um Ihr Konzept zu testen, baut Ihr Team erst mal ein Funktionsmuster in einem kleinen Massstab.



Abbildung 1: Logo der Firma «LineXpress – we deliver along the line»

Sie werden also am Ende des PM2-Unterrichts mit Ihrem Team einen Roboter gebaut haben, welcher in der Lage ist, bei einem Haus Pakete aufzunehmen, entlang einer Linie zu einem entsprechenden Haus zu fahren und dort das Paket wieder abzuliefern. Da in diesem Marktumfeld grosse Konkurrenz herrscht, muss das Ganze natürlich so schnell wie möglich geschehen. Neben einer ausgeklügelten Mechanik und einer gut funktionierenden Software geht es also auch um eine gute Strategie, damit der Roboter die Aufgabe schnell und effizient erledigt.



Abbildung 2: AI-generiertes Bild eines Paket-Roboters, welcher einer Linie entlangfährt

Weitere Bestimmungen zur Challenge finden Sie weiter unten in den Abschnitten "Roboter" und "Spielfeld".

Durchführung:

Bei dieser Übung handelt es sich um ein Entwicklungsprojekt gemäss den VDI-Richtlinien 2221/2222. Sie werden selbstständig Teams von 5 bis 7 Leuten bilden (je nach Klassengrösse). Die Übung muss gemäss dem im PM1 erlernten Vorgehen für Entwicklungsprojekte umgesetzt werden (mehr dazu unten). Die Übung ist ergebnisoffen. Das bedeutet, dass es nicht ‚DIE‘ Lösung gibt, an welcher Sie gemessen werden. Neben der Lösung (Kreativität und Funktion) werden Sie an der angewandten und sichtbar gemachten Methodik und der Qualität der abgegebenen Unterlagen und Ergebnisse gemessen sowie an der Video-Präsentation und dem Projektbericht. Bewertet werden also Prozess und Produkt.

Im Folgenden finden Sie die Rahmenbedingungen für den Roboter und die Umgebung, in der sich der Roboter bewegen muss. Lesen Sie die Bedingungen genau durch.

Ganz wichtig: Ziel der Challenge ist nicht nur die schnellste Zeit, sondern ein robustes, gut dokumentiertes, methodisch entwickeltes Gesamtsystem.

Challenges:

Sie starten im Depot der «LineXpress – we deliver along the line»-Firma. Dort darf Ihr Roboter eine maximale Grösse von 200 x 200 x 200 mm haben. Danach fahren Sie im Uhrzeigersinn entlang der schwarzen Line bis zum ersten Haus, welches mit einer Querlinie am Boden markiert ist. Dort befindet sich auch ein farbiges Feld am Boden, mit welchem Sie erkennen können, in welcher der vier verschiedenen Positionen sich das Paket beim Haus befindet (die Zuordnung der Farben zu den jeweiligen Häusern wird sich zwischen Übungs-Spielfeld und finalem Spielfeld ändern). Die Paketposition kann dabei im Abstand zur Querlinie am Boden in x- und y-Richtung variieren sowie auf der Terrasse stehen, womit das Paket etwas erhöht ist. Die genauen Positionen sind weiter unten definiert.

Sie nehmen das erste Paket auf und entscheiden, je nach Ihrer Strategie, ob Sie zuerst das Paket beim entsprechenden Haus auf der anderen Seite abliefern, oder ob Sie zuerst alle Pakete aufsammeln und dann alle Pakete abliefern. Dabei ist es wichtig, dass Sie das richtige Paket am richtigen Ort abliefern. Deshalb sind die Pakete in derselben Farbe codiert wie das Feld am Boden des Absenders und des Empfängers.

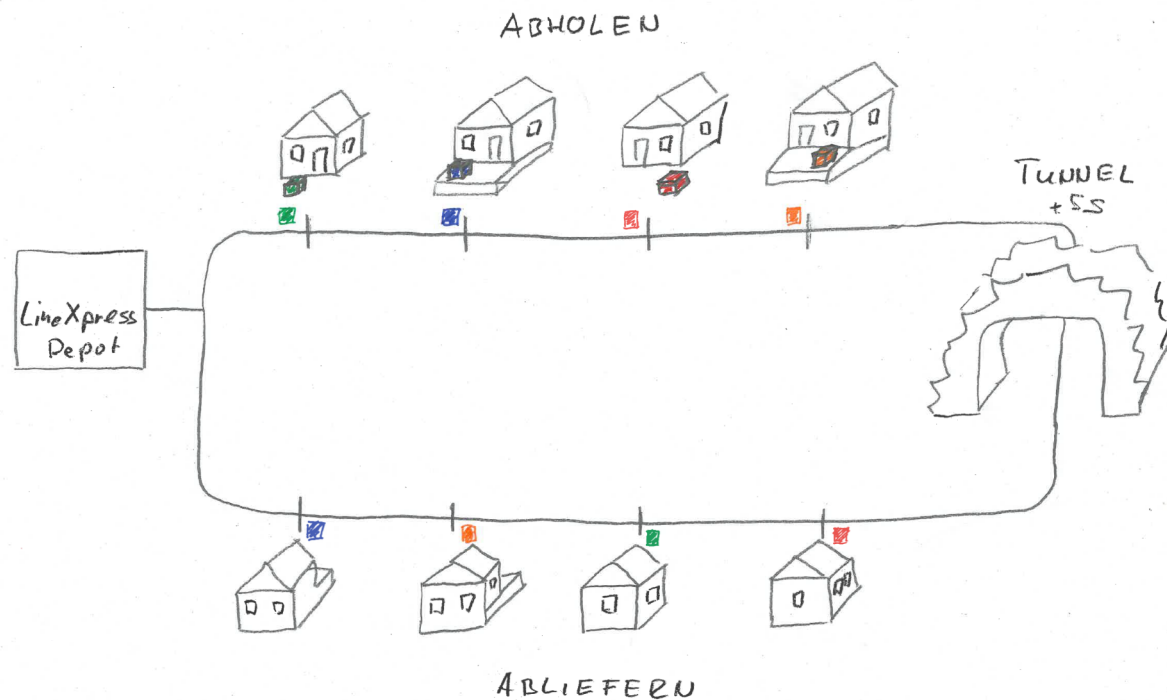


Abbildung 3: Spielfeld

Um die Pakete abzuliefern, können Sie entweder weiter im Uhrzeigersinn fahren, wobei Sie durch den Tunnel fahren müssen, welcher einen Durchgang von 250 x 250 mm hat und der Sie eine Zeitstrafe von 5 Sekunden kostet, da die Beleuchtung im Tunnel sehr schlecht ist und die Sicherheit für Ihren Fahrer und die Pakete nicht gewährleistet ist. Wenn Sie 180° umdrehen und so zu den Abliefer-Häuser fahren, haben Sie unter Umständen einen weiteren Weg, müssen zweimal um 180° drehen, müssen aber nicht durch den Tunnel und sparen so die 5 Sekunden Zeitstrafe. Die Abliefer-Häuser sind wiederum mit einem schwarzen Strich (kürzer als bei den Abhol-Häusern) am Boden markiert. Bei den Abliefer-Häusern muss das Paket analog zur Position bei den Abholhäusern deponiert werden. Sobald das letzte Paket richtig abgeliefert wurde, wird die Zeit gestoppt.

Roboter:

- Der Roboter muss am Anfang eine maximale Grösse von 200 x 200 x 200 mm aufweisen.
- Der Roboter startet im Depot, ohne Paket innerhalb des markierten Bereichs.
- Die Farbcodierung der Häuser und somit der Pakete ist zufällig, dies müssen Sie entsprechend im Logikteil der Software berücksichtigen (keine hardcodierten Zuordnungen von Paketen und Häusern).
- Der Roboter muss über einen Linien-Folge-Sensor und einen Farbsensor (wird weiter unten beschrieben) verfügen.
- Der Roboter muss in der Lage sein, die Pakete an 4 verschiedenen und klar definierten Positionen aufzunehmen und wieder entsprechend an derselben Position abzuliefern.
- Falls Sie durch den Tunnel fahren wollen, darf Ihr Roboter dabei natürlich nicht grösser als 250 x 250 mm im Querschnitt sein.

- Ob Sie die Pakete auf dem Roboter zwischenlagern oder nach jeder Aufnahme abliefern ist Ihnen überlassen.

Pakete:

- Die Pakete haben eine Grösse von 30 x 30 x 30 mm.
- Die Pakete haben seitlich eine Nut, damit sie mit einer Gabel aufgenommen werden können.
- Die Pakete haben auf der oberen Seite einen kleinen Magneten, mit welchem das Paket aufgenommen werden kann.
- An der unteren Seite haben die Pakete einen deutlich stärkeren Magneten, damit sie bei den Abliefer-Häusern in Position bleiben, wenn Sie den Greifer wieder wegbewegen.
- Eine gültige Ablieferung der Pakete ist gegeben, wenn der Magnet des Pakets durch den Magnet des Hauses gehalten wird.

Hier kommt ein 3D-Bild eines Pakets mit den relevanten Massen.

Spielfeld:

- Es werden sowohl 4 Abhol- wie auch 4 Abliefer-Häuser vorhanden sein.
- Das Depot der Firma «LineXpress – we deliver along the line» wird mit einem dünnen Rahmen am Boden markiert.
- Alle Fahrlinien (Strasse und Querbalken) haben eine Breite von 20 mm und sind schwarz.
- Die Querlinien der Abhol-Häuser sind **100** mm breit und diejenigen der Abliefer-Häuser sind **50** mm breit.
- Jedes Haus ist mit einem farbigen Feld von 40 x 40 mm markiert.
- Die Position der farbigen Felder entnehmen Sie den Bildern weiter unten.
- Die Positionen der Pakete gegenüber den Querlinien entnehmen Sie bitte auch den Bildern weiter unten.

Hier kommt ein Bild des Spielfelds

Abbildung 4: Spielfeld von oben

Hier kommt ein Bild der farbigen Felder relativ zu den Querbalken

Abbildung 5: Position der farbigen Markierfelder

Hier kommt ein Bild der Positionen der Pakete relativ zu den Querbalken

Abbildung 6: Positionen der Pakete gegenüber den Querbalken

Zeitmessung:

- Die Zeit startet, sobald sich der Roboter im Depot zu bewegen beginnt.
- Die Zeit stoppt, sobald der Roboter beim letzten Abliefer-Haus einen Zustellversuch gemacht hat (Ein Zustellversuch gilt als erfolgt, sobald der Roboter das Paket losgelassen hat oder der Greifer eine definierte Ablagebewegung ausführt).

- Pro Durchfahrt des Tunnels wird eine Zeitstrafe von 5 Sekunden gegeben.
- Jedes Paket, welches nicht im Zielfeld des jeweiligen Ablieferer-Hauses abgeliefert wurde, gibt eine Zeitstrafe von 20 Sekunden.

Weitere Regeln:

- Die Wahl der Strategie (zuerst alle Pakete aufnehmen oder einzeln abliefern) hat keinen Bonus oder Malus zur Folge.
- Schafft der Roboter die korrekte Ablieferung im ersten Lauf, so wird ein Bonus von 20 Sekunden gegeben.
- Jedes Team hat zwei Durchgänge zur Verfügung, falls der erste Versuch erfolgreich ist, wird kein zweiter Durchlauf gewährt.
- Funktioniert der Roboter nach einem Neustart immer noch nicht wie gewünscht, kann am Schluss (wenn alle anderen Teams fertig sind) noch einmal ein Versuch gemacht werden. Dies wird allerdings mit einer Zeitstrafe von 20 Sekunden gewertet.
- Es darf während des Wettkampfs in keiner Art und Weise mit dem Roboter kommuniziert werden. Der Roboter muss die Aufgabe vollständig autonom erledigen.
- Bei Bedarf können kleinere Regeländerungen oder -erweiterungen während des Semesters vereinbart werden.
- Werden Häuser oder der Tunnel verschoben, hat dies eine Zeitstrafe von 30 Sekunden pro verschobenem Objekt zur Folge.
- Ein Lauf wird nach maximal 8 Minuten abgebrochen und als nicht erfolgreich gewertet.

Elektronik:

Die abgegebene Elektronik besteht zum einen aus einem Mikrocontroller-Board (Nucleo-Board F446RE) und zum anderen aus einem Motherboard.

Das Nucleo-Board ist eine Platine, welche einen ARM-Mikrocontroller und die dazu benötigte Peripherie besitzt. Diese Nucleo-Boards lassen sich mittels der sogenannten Mbed Programmierplattform in C++ programmieren.

Das von uns entwickelte Motherboard stellt die restliche Peripherie zur Verfügung, welche typischerweise für den Bau von einfachen mobilen Robotern benötigt wird. Dazu gehören:

- 1 IMU (Inertial Measurement Unit mit Gyro, Beschleunigungssensoren und Magnetometer)
- 3 Motorentreiber für die direkte Ansteuerung von DC-Motoren
- 3 Encoder-Counter, um die Umdrehungszahl der DC-Motoren einzulesen
- 4 digitale I/O, 3.3V (5V tolerant)
- 4 analoge Eingänge, 3.3V (5V tolerant)
- Stecker, um die Motoren, Encoder, Sensoren anzuschliessen
- Ladebuchse und einfache Spannungsanzeige für den Akku.
- 1 Slot für SD-Karte, um einfach Daten zu loggen oder auszulesen

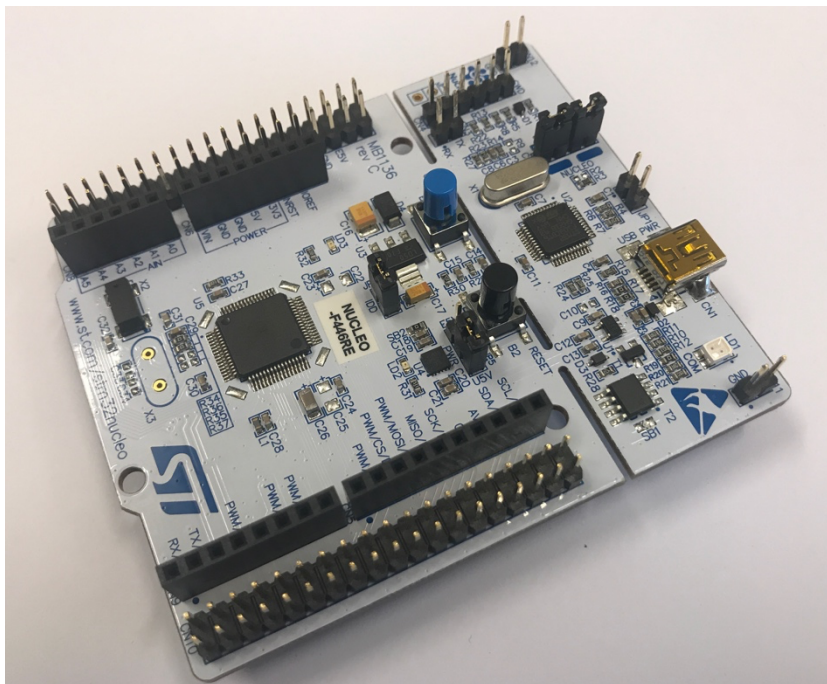


Abbildung 7: Nucleo-Board

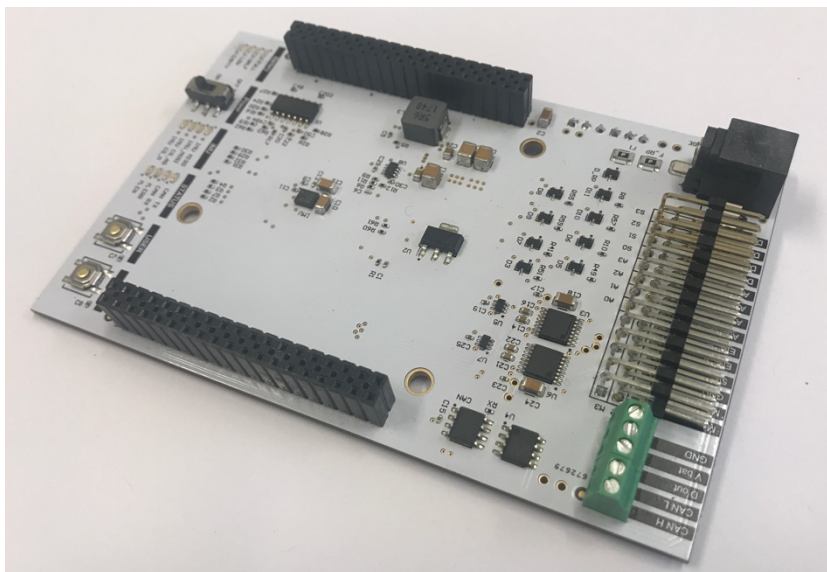


Abbildung 8: ZHAW-Motherboard

Zudem erhält jedes Team 1 Akkupaket. Dabei handelt es sich um 2 in Serie geschaltete NiMH-Akkus mit jeweils 5 Zellen. Sie haben also ein 12 V-Akkupaket mit 2300 mAh.

Detailliertere Informationen über die Elektronik wird Ihnen im Verlauf der Vorlesung gegeben.

Selbstverständlich darf nur so viel Hardware eingesetzt werden, wie auch tatsächlich vom Board kontrolliert werden kann.

Aktuatoren and Sensoren:

Im Folgenden wird die Hardware beschrieben, welche jedem Team abgegeben wird:

1 X Ultraschall-Sensor

https://wiki.seeedstudio.com/Grove-Ultrasonic_Ranger/

2 x DC-Motor mit Encoder (Getriebeübersetzung variiert):

Motor: <https://www.pololu.com/category/167/20d-metal-gearmotors>

Encoder: <https://www.pololu.com/product/3499>

2 x RC-Servos:

<https://www.conrad.ch/de/p/reely-standard-servo-cys-s0090-analog-servo-getriebe-material-metall-stecksystem-jr-2203091.html>

<https://www.modellsport.ch/RC-Elektronik/Servo/Futaba/Futaba-Servo-S-3001.html>

1 x analoger Distanzsensor:

<https://www.digikey.ch/de/products/detail/sharp-socle-technology/GP2Y0A41SK0F/3884447>

1 x digitaler Endschalter:

<https://www.digikey.ch/de/products/detail/omron-electronics-inc-emc-div/D3V-166-2C5/5236792>

1 x LED:

<https://www.conrad.ch/de/p/barthelme-led-sortiment-rot-blau-gruen-gelb-warmweiss-rund-5-mm-120-mcd-800-mcd-2000-mcd-20000-mcd-30-35-20-1666905.html>

1 x Line-Follower-Array

<https://www.sparkfun.com/products/13582>

1x Farbsensor

<https://www.berrybase.ch/tcs230-tcs3200-farbsensor-modul>

<https://elektro.turanis.de/html/prj029/index.html>

Budget:

Für zusätzliche Hardware steht jedem Team ein Budget von CHF 75.- zur Verfügung. Es wird empfohlen, lediglich Komponenten zu bestellen, welche mit den bestehenden Softwaretreibern verwendet werden können. Genauere Informationen zu möglichen Lieferanten und dem Bestellprozess werden zu gegebener Zeit kommuniziert. Weiter ist darauf zu achten, dass – sofern möglich – Komponenten verwendet werden, welche bereits im Fundus vorhanden sind (auch dazu später mehr). Diese Fundus-Komponenten werden virtuell zu 50 % dem Budget belastet.

Abzugebende Dokumente:

Sie verfassen einen Projektbericht und geben diesen elektronisch ab. Folgende technischen Inhalte müssen vorhanden sein:

Produktentwicklung:

- Einführung / Übersicht
- Zeitplan (→ im Anhang)
- Kurzer Recherchebericht
- Anforderungsliste
- Funktionsstruktur (→ im Anhang)
- Morphologischer Kasten (→ im Anhang)
- Darstellung der wichtigsten Lösungsprinzipien mit aussagekräftigen und sauberen grobmassstäblichen Skizzen
- Nutzwertanalyse
- Kurz-Budget mit Zusammenstellung der Auslagen (Teile aus Fundus 0.5 x, alle 3D-Druckteile, Laserteile, Schrauben, Kabel... gratis) (→ im Anhang)
- Evtl. von NWA zu finalem Konzept, falls finale Lösung sich stark von theoretischer Lösung unterscheidet.

Mechanik:

- Einführung / Übersicht
- Ausarbeitung der mechanischen Lösung (von der Idee zum Prototyp)
- Berechnungen von Motorenmomenten oder ähnlichem
- Skizzen der finalen Lösung
- Screenshots der CAD-Konstruktion
- Evtl. etwas über den Zusammenbau

Elektronik:

- Einführung / Übersicht
- Beschreibung selbst hergestellte Elektronik (nur wenn selbst Elektronik entwickelt wurde)
- Vereinfachtes Elektronikschemata, das zeigt, welche Ein- und Ausgänge an welchem Pin hängen)
- Dokumentation über verwendete Bauteile inkl. Motoren, Sensoren...

Software:

- Einführung / Übersicht
- Flow-Chart Diagramm zur Implementierung ggf. erweitert mit Textbeschreibung

Diskussion und Ausblick:

Dieser Abschnitt bespricht die erzielten Ergebnisse bezüglich ihrer Erwartbarkeit, Aussagekraft und Relevanz. Er gibt einen Rückblick auf die Aufgabenstellung, ob sie erreicht bzw. nicht erreicht worden ist; er enthält ein Fazit über das Projekt sowie über mögliche Verbesserungen.

Insbesondere wird auch ein detaillierter Vergleich der gebauten Lösung mit der Anforderungsliste erwartet (sind alle Anforderungen erfüllt, wie gut sind die Wünsche erfüllt).

Verschiedenes im Anhang:

- Protokoll der wesentlichen Tests und der daraus resultierenden Erkenntnisse
- Sensorkalibrierung
- Eigene Dokumente oder Daten, wie z. B. Zeitplan

Der Bericht soll so kurz wie möglich und so lang wie nötig gehalten werden. 10 Seiten reiner Text sollten nicht überschritten werden. Der Anhang und Abbildungen zählen nicht dazu.

Weitere Details zum Bericht werden Sie während des Semesters von Frau Runte erhalten.

Zusätzlich ist elektronisch abzugeben:

- .stp der Konstruktion
- Elektronikschaltungen (falls vorhanden)
- Kompletter Softwarecode
- Video

Neben dem Bericht werden Sie Ihre Ergebnisse in der Semesterwoche 13 in Form einer Video-Präsentation vorstellen. Dazu erhalten Sie während des Semesters detailliertere Informationen.

Termine

- Start der Übung U2 ist die SW 1
- Die Video-Präsentationen müssen in SW 13 (13.5. (Klasse ST25a) und 15.5. (Klasse ST25t)) abgegeben werden.
- Die Berichte müssen in SW 14 am Abend des 23.5. abgegeben werden.
- In der letzten oder zweitletzten Woche wird ein Wettkampf stattfinden, bei welchem die Roboter den Parcours abfahren müssen. Der Termin wird noch kommuniziert.
- Der PM2-Semesterplan ist zu beachten.
- Es werden zwei Projektreviews gemäss Semesterplan sowie Coachinggespräche (im Team) im Rahmen der Förderung der ‚Non-Technical Skills‘ durchgeführt.
- In der ST25t fallen 2 Unterrichtseinheiten aus (Karfreitag und 1. Mai).
- Die Soll-Arbeitsstunden pro Person betragen 56 Lektionen (4x14Lektionen) während des Unterrichts plus dieselbe Zeit an Selbststudium, also insgesamt 112h. Das ergibt für eine 6er-Gruppe 672h. Bitte berücksichtigen Sie diese Soll-Zeit in Ihrer Zeit- und Arbeitsplanung.

Modul PM2

Modulstruktur und Leistungsnachweise des Moduls PM2 sind gemäss der Schulverwaltungssoftware Evento:

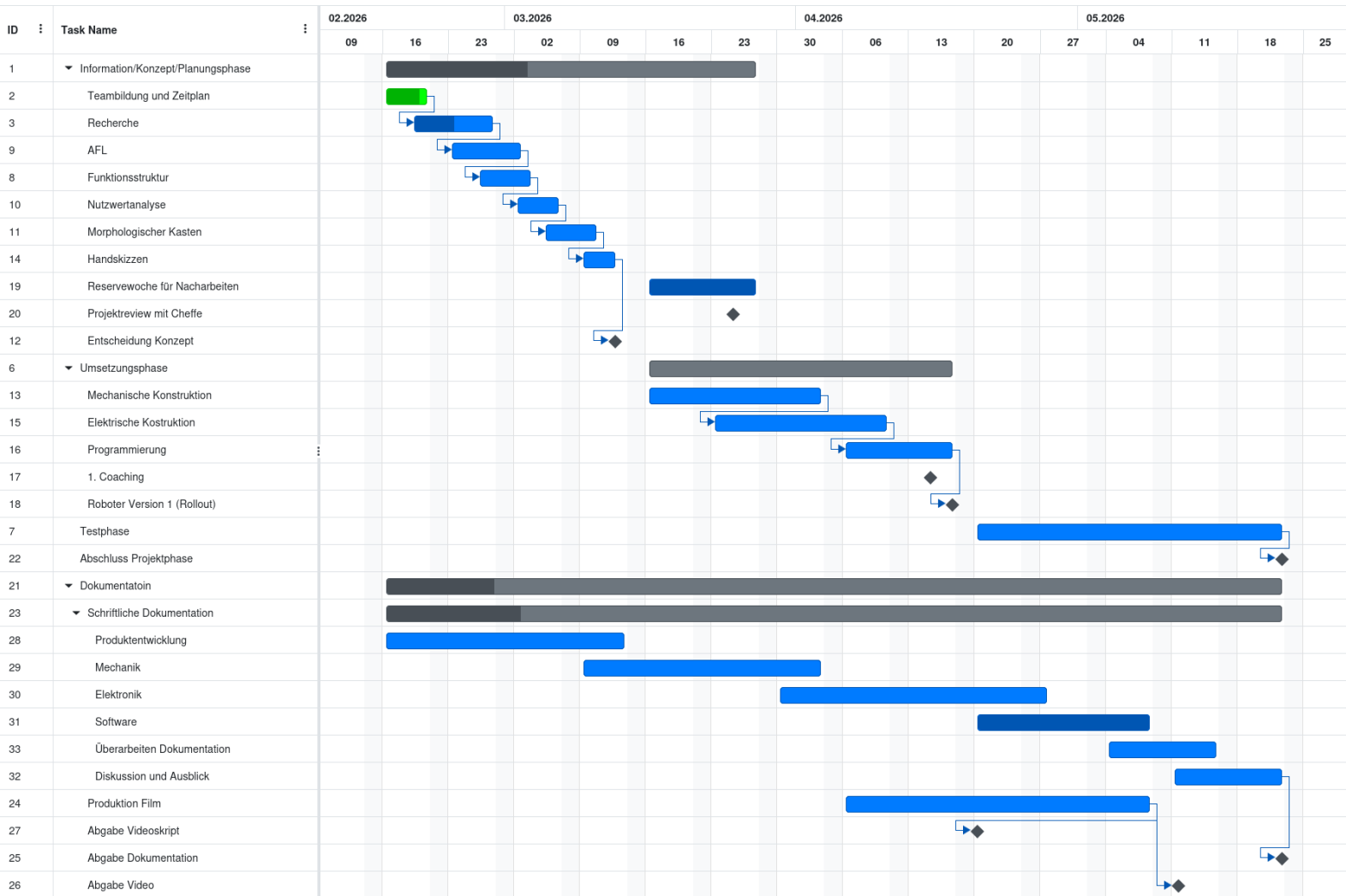
Bezeichnung	Art	Form	Umfang	Bewertung	Gewichtung
Leistungsnachweise während Studiensemester	Video-Präsentation	mündlich	tbd	Benotung	30%
Projekt	Projekt U2	schriftlich	Semesterarbeit	Benotung	70%

Die Video-Präsentation sowie das Projekt U2 inkl. Projektbericht ergeben eine Teamnote und werden innerhalb des Semesters erarbeitet.

Video-Präsentation: Die detaillierten Angaben zur Video-Präsentation werden Sie im Verlauf des Semesters noch erhalten.

Projekt U2: Bewertet wird insbesondere das Produkt (die gefundene und umgesetzte Lösung) und der Prozess anhand der abgegebenen Unterlagen gemäss Auflistung ‚Abzuliefernde Ergebnisse‘ und der Beobachtungen während des Semesters. Im Vordergrund steht, dass die Teams eine kreative Lösung finden. Das Abschneiden am Wettkampf fliesst nur minimal in die Beurteilung ein, ist aber ein Teil davon, da dieses eine gute Aussage über das Erfüllen der Aufgabenstellung macht.

Zeitplan



Powered by: onlinegantt.com

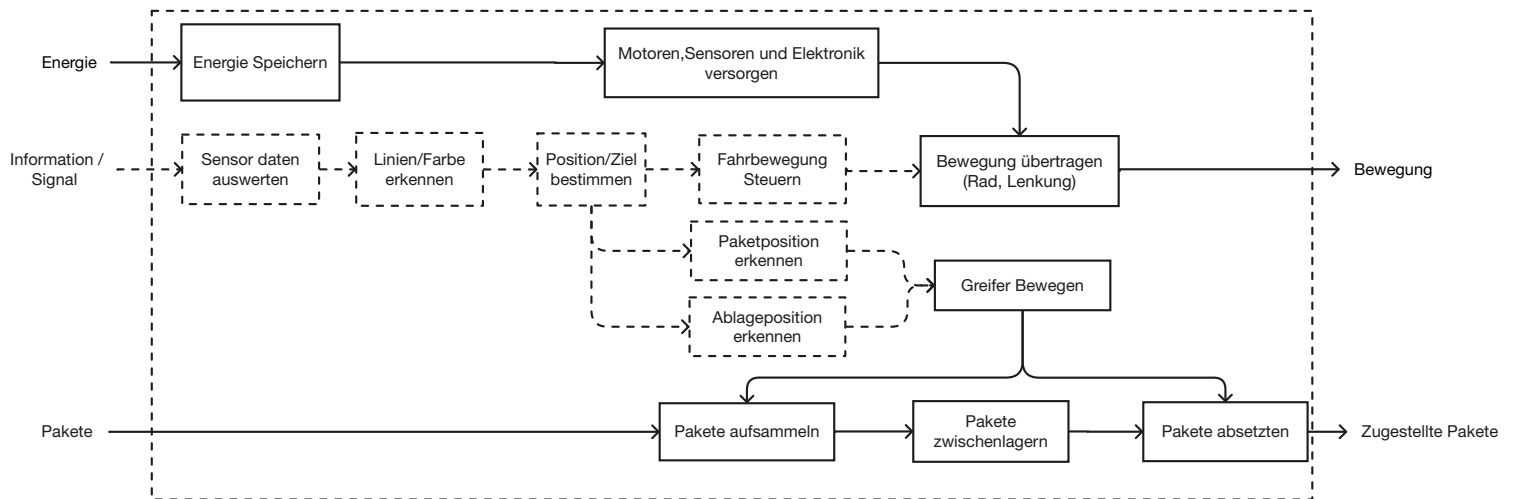
5/13/2026		Anforderungsliste für Paketroboter		F = Forderung W = Wunsch
		Projekt: MoonCarrier		Team 3
Anforderungen				
F, W	Nr.	Bezeichnung	Werte, Daten, Erläuterungen, Änderungen	Verantwortlich, Klärung durch:
	1	Funktion		
F	1.01	Pakete einsammeln/aufnehmen	Aufnahme von 30 × 30 × 30 mm Paketen	
W	1.02	Zwischenlagerung der Pakete auf dem Roboter	evtl. Für 4 Pakete gleichzeitig	
F	1.03	Ablieferung der Pakete	Richtige Positionierung bei der Abgabe	
F	1.04	Spurhaltung	Fahrzeug muss auf der Spur bleiben und an den Stopps halten	
F	1.05	Wiederholbarkeit	Durchläufe mit geringen Fehlern	
F	1.06	Vollständig autonom	keine Kommunikation während Lauf	
F	1.07	Paketpositionierung	Variieren in x- und y-Richtung, Können erhöht auf Terrasse stehen	
F	1.08	Betriebsdauer	genug Akku für mindestens 2 Durchläufe	
W	1.09	Intelligente Routenstrategie	Effizienteste Route	
	2	Konstruktion		
F	2.01	Roboteranfangsgrösse	Grösse <=200x200x200mm	
F	2.02	Robotergrösse bei Tunneldurchquerung	Querschnittgrösse <= 250x250mm	
W	2.03	Antrieb	180° Drehungen auf der Strecke möglich	
F	2.04	Fahrwerk	Genügend Traktion für Beschleunigung, Bremsen.	
W	2.05	Fahrstabilität	Schwerpunkt tiefer als der Mittelpunkt	
F	2.06	Nucleo-Board F446RE	1x	
F	2.07	Ultraschall-Sensor	1x	
F	2.08	DC-Motor mit Encoder (Getriebeübersetzung variiert)	2x	
F	2.09	RC-Servos	2x	
F	2.10	analoger Distanzsensor	1x	
F	2.11	digitaler Endschalter	1x	
F	2.12	LED	1x	
F	2.13	Line-Follower-Array	1x	
F	2.14	Farbsensor	1x	
W	2.15	Filament	Eco	
W	2.16	ansprechendes Design	Sauber verarbeitet, Komponenten verstaut unter Abdeckung	
W	2.17	optimiertes Fahrgestell	verbesserte Spurhaltung zur Minimierung von Korrekturbewegungen	
W	2.18	Farbschema	LineXPRESS-Logo (Schwarz/Orange)	
W	2.19	Einfacher Zusammenbau	Bauteile einzeln austauschbar, um defekte Teile einfach auszutauschen	

	3	Wettkampfregeln	
F	3.01	Spielfeld	4 Abholhäuser, 4 Ablieferhäuser, Tunnel und Depot
F	3.02	Vollständig autonom	keine Kommunikation während Lauf
F	3.03	Korrekte Zustellung	Das Paket muss bei der gleichen Farbe abgelegt werden, wie es aufgehoben wurde.
F	3.04	Tunnel-Regeln	Durchgang: 250 × 250 mm, ede Durchfahrt: +5 Sekunden Zeitstrafe, Alternative: 180° drehen und ohne Tunnel zu durchfahren
F	3.05	Zeitmessung	Start: Sobald sich der Roboter im Depot bewegt Stopp: Nach letztem Zustellversuch
W	3.06	Zeitstrafen vermeiden	Tunneldurchfahrt: +5 s Falsch zugestelltes Paket: +20 s pro Paket Verschieben von Haus oder Tunnel: +30 s pro Objekt Zusätzlicher Reparatur-Versuch am Ende: +20 s Lauf > 8 Minuten: Abbruch, nicht erfolgreich
F	3.07	Durchläufe	2 Versuche pro Team Wenn 1. Lauf erfolgreich → kein 2. Lauf
W	3.08	Bonus	Wenn im ersten Lauf alles korrekt funktioniert → -20 Sekunden Bonus
	4	Software	
F	4.01	Strukturierte Softwarearchitektur	Übersichtlich / Kommentiert
W	4.02	Fehler-Logging über LED-Codes	Fehler klar erkennbar
W	4.03	Parametrierbarkeit	PID-Werte / Schwellenwerte anpassbar ohne Code-Umbau.
	5	Termine	
F	5.01	Projektstart	Start des Projekts ist die SW 1
F	5.02	Abgabe Videoskript	Dokument Video-Skript ist bis Mo 20.4 auf Moodle abgeben
F	5.03	Abgabe Dokumentation	Die Berichte müssen in SW 14 am Abend des 23.5. abgegeben werden.
F	5.04	Abgabe Video	Die Video-Präsentationen müssen in SW 13 (13.5. (Klasse ST25a)) abgegeben werden.
	6	Dokumente	
F	6.01	Produktentwicklung	Zeitplan Recherche Anforderungsliste Funktionsstruktur Morphologischer Kaste Grobmassstäbliche Handskizze Nutzwertanalyse Handskizze, CAD-Datei und Screenshot der finalen Lösung

Team 3

	7	Normen	
F	7.01	VDI-Richtlinien 2221/2222	
	8	Robustheit & Sicherheit	
W	8.01	Stoßfestigkeit	Teile brechen nicht ab bei Kollision mit Haus
W	8.02	Keine scharfen Kanten	Alle Kanten abgerundet
W	8.03	Magnetische Störeinflüsse minimieren	Pakete fallen nicht ungewollt vom Magneten
W	8.04	Stabil bei Fehlmanöver	Kein Umkippen wenn der Arm im falschen Moment ausfährt

Funktionsstruktur



Recherche – Moon Carrier

Team: 3

Inhaltsverzeichnis

1	Einleitung	2
1.1	Ausgangslage	2
1.2	Unterteilung	5
2	Mechanik	6
2.1	Geometrie Line-Follow	6
2.2	Antriebskonzept	7
2.3	Greif- und Armmechanismus	7
2.3.1	Armgeometrie	7
3	Elektronik	7
3.1	NUCLEO-F446RE	7
3.2	ZHAW PES Board	8
3.2.1	DC-Motor	8
3.2.2	Servo-Motor	8
3.2.3	LED	8
3.2.4	Drucksensor	8
3.2.5	IR-Sensor	9
3.2.6	Ultraschall-Sensor	9
3.2.7	Farbsensor	9
3.2.8	Line-Follower-Array	9
4	Informatik	10
4.1	PD-Regler (Linienfolger)	10
4.2	Zustandsmaschine	10
5	Vergleichbare Projekte	10

1 Einleitung

1.1 Ausgangslage

Im Rahmen des Moduls PM2 entwickelt Team 3 einen autonomen Lieferroboter namens *LineXpress*. Dieser soll auf einem definierten Spielfeld entlang einer schwarzen Fahrlinie navigieren, farblich kodierte Pakete aufnehmen und diese den entsprechend markierten Ablieferhäusern zuordnen. Dabei müssen sowohl mechanische, elektronische als auch softwareseitige Anforderungen erfüllt werden.

Die wesentlichen Randbedingungen der Aufgabenstellung sind nachfolgend zusammengefasst:

- **Roboter-Masse:** 200 mm × 200 mm × 200 mm (maximale Abmessungen)
- **Tunnel:** Durchmesser 250 mm × 250 mm; Durchfahren ohne Berühren vermeiden, da sonst 5 s Zeitstrafe
- **Pakete:** 30 mm × 30 mm × 30 mm; kleiner Magnet auf der Oberseite, grosser Magnet auf der Unterseite
- **Gültige Ablieferung:** Das Paket muss vollständig vom Haus gehalten werden
- **Fahrlinie:** 20 mm breit, schwarz auf hellem Untergrund
- **Querlinien Abhol-Häuser:** 100 mm breit, schwarz
- **Querlinien Abliefer-Häuser:** 50 mm breit, schwarz
- **Hausfeld:** 50 mm × 40 mm, farblich analog zum zugehörigen Paket

Die Pakete befinden sich auf dem Spielfeld an definierten Positionen und sind farblich (rot, blau, grün, gelb) den jeweiligen Ablieferhäusern zugeordnet. Die genauen Verschiebungen der Pakete relativ zur Fahrlinie sind in der folgenden Tabelle aufgeführt:

Farbe	Horizontale Versch. [mm]	Vertikale Versch. [mm]	Höhe über Spielfeld [mm]
Rot	25	105	4
Blau	145	105	14
Grün	145	105	4
Gelb	25	105	14

Tabelle 1: Paketpositionen auf dem Spielfeld

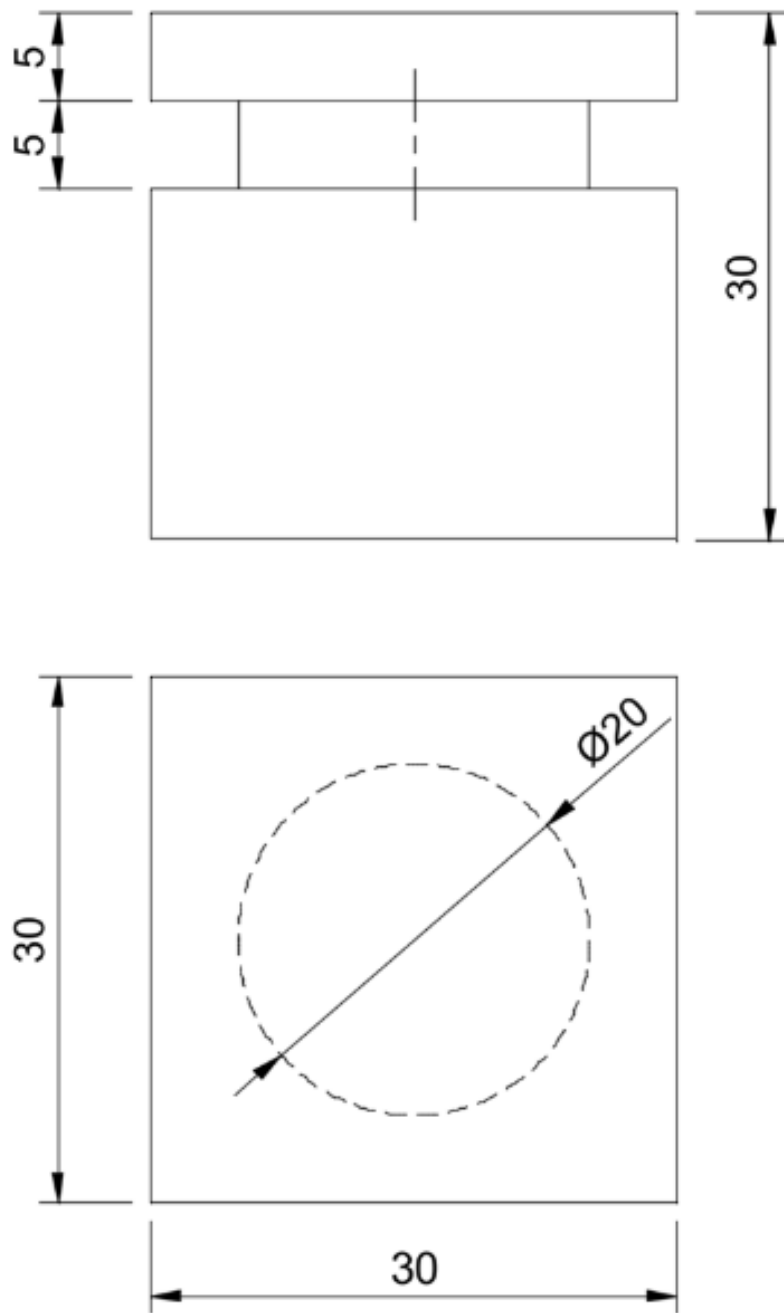


Abbildung 1: Bemassung Paket – Quelle: Aufgabenstellung

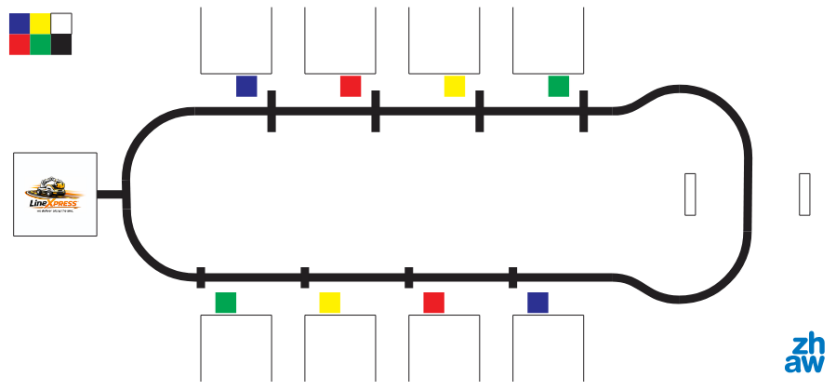


Abbildung 2: Spielfeld – Quelle: Aufgabenstellung

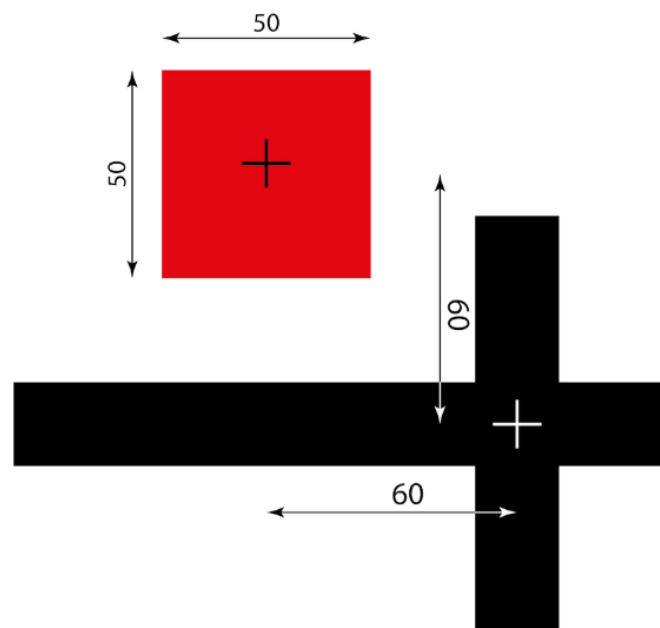


Abbildung 3: Position Farbfeld – Quelle: Aufgabenstellung

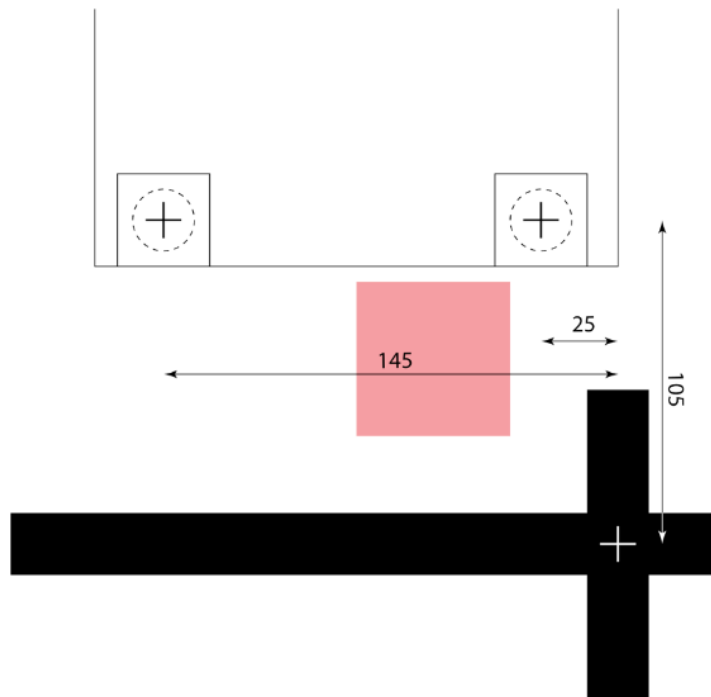


Abbildung 4: Position Paket – Quelle: Aufgabenstellung

1.2 Unterteilung

Jedes mechatronische System kann in drei Hauptsegmente unterteilt werden: Mechanik, Elektronik und Informatik. Diese drei Bereiche sind eng miteinander verknüpft und beeinflussen sich gegenseitig, wie in Abbildung 5 dargestellt.

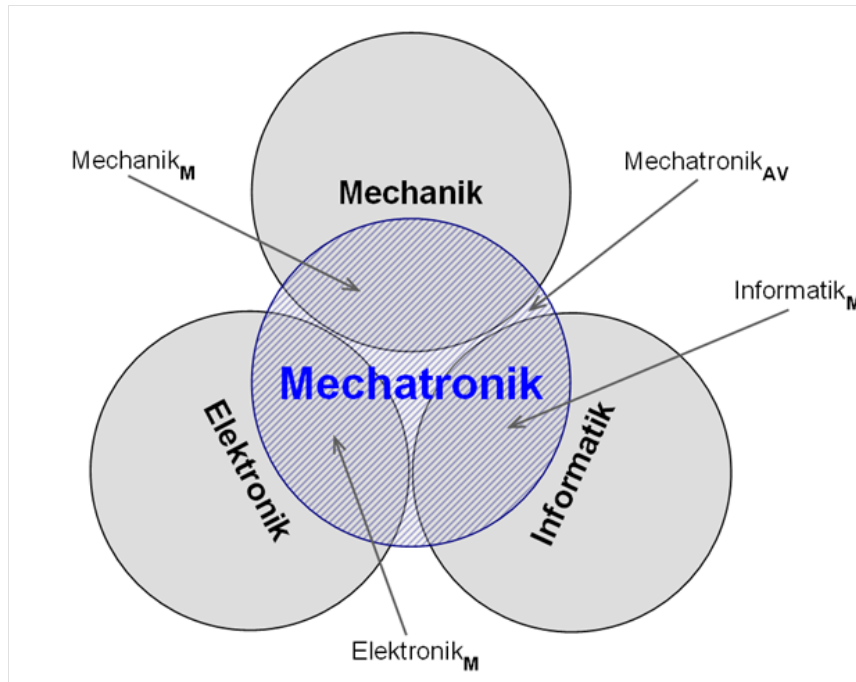


Abbildung 5: Mechatronic – Quelle: Wikipedia

In dieser Recherche werden für jedes dieser Segmente mögliche Optionen, Bauteile und Lösungsansätze untersucht. Dementsprechend ist dieses Dokument in drei Kapitel gegliedert.

Die Recherche dient anschliessend als Grundlage zur Entwicklung weiterer Konzepte, beispielsweise in Form eines morphologischen Kastens.

Da wir mit Rapid Prototyping arbeiten, bleibt die Recherche dynamisch. Dieses Dokument wird fortlaufend weiterentwickelt, und neue Ideen sowie gewonnene Erkenntnisse fliessen kontinuierlich ein.

2 Mechanik

2.1 Geometrie Line-Follow

Für einen zuverlässigen Linienfolgealgorithmus ist die geometrische Anordnung des Liniensensors relativ zur Radachse (Wheelbase) entscheidend [9]. Der Sensor muss zwingend *vor* der Radachse montiert sein, damit der Regler frühzeitig auf Kurven reagieren kann und nicht erst dann, wenn der Roboter bereits von der Linie abgewichen ist.

Je grösser der Abstand zwischen Sensor und Radachse, desto mehr Zeit hat der Regler, gegenzusteuern. Allerdings führt ein zu grosser Abstand bei engen Kurven zu Überschwingern, da die Regelabweichung zu früh und zu stark korrigiert wird. Ein praxiserprobter Kompromiss liegt bei einem Vorhalteabstand von ca. 5–10 cm vor der Vorderachse.

Zudem ist auf eine symmetrische Montage des Sensorarrays zu achten, damit keine systematische Drift in eine Fahrtrichtung entsteht.

2.2 Antriebskonzept

Als Antriebskonzept würde sich ein Differentialantrieb (Differential Drive) sehr gut eignen. [2]. Dabei werden zwei unabhängig angesteuerte Antriebsräder auf einer gemeinsamen Achse verwendet. Durch unterschiedliche Drehzahlen der Räder kann der Roboter kurven und sich auf der Stelle drehen. Dieses Konzept ist mechanisch einfach, gut regelbar und erprobt für Linienfolger-Anwendungen.

Ein drittes Stützrad (Casterwheel) oder ein Gleitklotz verhindert das Kippen und gewährleistet eine stabile Dreipunktauflage auf dem Spielfeld.

2.3 Greif- und Armmechanismus

Für die Aufnahme und Abgabe der Pakete wird ein Greif- bzw. Armmechanismus benötigt. Da die Pakete auf ihrer Unterseite einen grossen Magneten tragen, bieten sich zwei grundlegende Ansätze an [14]:

- **Passiver Magnetgreifer:** Ein fest montierter Gegenmagnet oder eine Eisenplatte am Roboter zieht das Paket beim Unterfahren automatisch an. Kein Aktuator nötig – einfach und zuverlässig, jedoch wenig kontrollierbar bei der Ablage.
- **Aktiver Servo-Greifer:** Ein RC-Servo bewegt einen Arm oder eine Klappe, die das Paket aufnimmt und kontrolliert ablegt. Dieser Ansatz ist komplexer, ermöglicht aber eine präzise und reproduzierbare Ablage direkt im Hausfeld [7].

2.3.1 Armgeometrie

Der Arm muss so dimensioniert sein, dass er innerhalb der maximalen Roboterausdehnung von 200 mm × 200 mm bleibt. Im eingeklappten Zustand darf der Arm die Tunneldurchfahrt (250 mm × 250 mm) nicht gefährden. Ein einfacher Ein-Gelenk-Arm, angetrieben durch einen einzelnen Servo, ist mechanisch unkompliziert und für diese Aufgabe ausreichend.

Der Servo wird so positioniert, dass er das Paket von oben greift oder von der Seite her einklappt. Die Endposition des Arms beim Abliefern muss so gewählt sein, dass das Paket vollständig auf dem Hausfeld abgestellt wird, was die Ablieferbedingung der Aufgabenstellung erfüllt.

3 Elektronik

3.1 NUCLEO-F446RE

Das STM32 Nucleo-F446RE ist das zentrale Steuerboard des Roboters. Es basiert auf dem ARM Cortex-M4 Mikroprozessor und läuft mit bis zu 180 MHz. Das Board bietet zahlreiche GPIO-Pins, UART-, SPI- und I²C-Schnittstellen sowie integrierte PWM-Kanäle, die für die Motoransteuerung genutzt werden. Die Programmierung erfolgt in C++ unter Verwendung der mbed-Bibliothek.

3.2 ZHAW PES Board

Das PES Board (Platform for Embedded Systems) der ZHAW ergänzt das Nucleo-Board und stellt zusätzliche Leistungselektronik bereit. Es beinhaltet u. a. Motortreiber (H-Brücken) für DC-Motoren und Servo-Ausgänge, was die externe Beschaltung erheblich vereinfacht.

3.2.1 DC-Motor

Als Antriebsmotoren werden DC-Getriebemotoren der Pololu 20D-Serie eingesetzt. Diese Motoren überzeugen durch ihr kompaktes Format (20 mm Durchmesser), verschiedene Getriebeuntersetzungen (z. B. 10:1 bis 150:1) und eine gute Verfügbarkeit von passendem Zubehör. In Kombination mit einem integrierten Encoder ermöglichen sie eine präzise Drehzahl- und Positionsregelung.

- **Motor:** Pololu 20D Metal Gearmotors [11]
- **Encoder:** Pololu Magnetic Encoder für 20D-Motoren [12]

Die Encoder liefern Impulse pro Umdrehung (CPR), aus denen die aktuelle Drehzahl sowie die zurückgelegte Strecke berechnet werden können. Dies ist Grundlage für die Odometrie und die Motorregelung.

3.2.2 Servo-Motor

RC-Servomotoren werden für die Greifmechanik oder den Paketablegemechanismus eingesetzt. Sie lassen sich einfach über PWM-Signale (typisch 50 Hz, Pulsweite 1–2 ms) ansteuern und sind in verschiedenen Drehmomentstufen erhältlich. Folgende Modelle wurden recherchiert:

- Reely Standard-Servo CYS-S0090 [13] – Analogservo mit Metallgetriebe, kompakt und robust
- Futaba Servo S-3001 [5] – Bewährtes Standardmodell, gutes Preis-Leistungs-Verhältnis

3.2.3 LED

LEDs werden zur Statusanzeige des Roboters verwendet (z. B. Bereitschaft, Fehler, Farberkennung). Es wird ein Sortiment mit roten, blauen, grünen und gelben LEDs (5 mm Durchmesser) eingesetzt, das auch warmweisse LEDs enthält. Die Lichtintensitäten variieren je nach Farbe zwischen 120 mcd und 20 000 mcd.

- Barthelme LED-Sortiment [1] – Verschiedene Farben und Helligkeiten

3.2.4 Drucksensor

Ein Drucksensor (Taster/Mikroschalter) kann zur Erkennung von Paketen oder als End-Anschlag-Sensor in der Greifmechanik eingesetzt werden. Der Omron D3V-166-2C5 ist ein kompakter Subminiatur-Mikroschalter mit definierten Schaltpunkten und langer Lebensdauer.

- Omron D3V-166-2C5 [10] – Mikroschalter, 0,1 A / 30 VDC

3.2.5 IR-Sensor

Der Sharp GP2Y0A41SK0F ist ein analoger Infrarot-Distanzsensor mit einem Messbereich von 4–30 cm. Er eignet sich zur Hinderniserkennung in kurzer Distanz, z. B. um das Vorhandensein eines Pakets zu detektieren oder einen Abstand zur Wand zu messen. Die Ausgangsspannung ist nicht-linear zur Distanz, weshalb eine Kalibrierung oder Lookup-Tabelle empfohlen wird.

- Sharp GP2Y0A41SK0F [15] – IR-Distanzsensor, 4–30 cm

3.2.6 Ultraschall-Sensor

Der Grove Ultrasonic Ranger von Seeed Studio ist ein einfach zu integrierender Ultraschallsensor mit einem Messbereich von 2–350 cm. Er kommuniziert über ein einzelnes digitales Pin (Trigger und Echo kombiniert) und ist daher GPIO-sparend. Der Sensor eignet sich für die Hinderniserkennung auf mittlere Distanz.

- Grove Ultrasonic Ranger [6] – Messbereich 2–350 cm, 3,3 V / 5 V kompatibel

3.2.7 Farbsensor

Zur Erkennung der Paket- und Hausfarben wird ein Farbsensor benötigt. Der TCS3200 (bzw. TCS230) ist ein weit verbreitetes Modul, das aus einem Array von Fotodioden mit verschiedenen Farbfiltern (Rot, Grün, Blau, kein Filter) besteht und die Lichtintensität als Frequenz ausgibt. Die Farbwerte werden aus dem Verhältnis der drei Kanäle berechnet.

- TCS3200 Farbsensor-Modul [17] – Einfache Ansteuerung, gute Dokumentation
- Beispielprojekt TCS3200 [4] – Referenzimplementierung mit Code-Beispielen

Es ist zu beachten, dass die Farberkennung stark von den Umgebungslichtverhältnissen abhängt. Eine gezielte Beleuchtung (z. B. mit weissen LEDs direkt am Sensor) verbessert die Reproduzierbarkeit erheblich.

3.2.8 Line-Follower-Array

Das QTR-8RC Reflectance Sensor Array von SparkFun enthält 8 IR-Reflexionssensoren in einer Reihe und ist speziell für Linienfolger-Anwendungen konzipiert. Es liefert pro Sensor einen digitalen oder analogen Wert, aus dem eine gewichtete Linienposition berechnet werden kann. Dieses Array vereinfacht die Sensorik erheblich gegenüber einzelnen Sensoren.

- SparkFun QTR-8RC Line Sensor Array [16] – 8 Kanäle, direkter Anschluss an Microcontroller-GPIO

4 Informatik

4.1 PD-Regler (Linienfolger)

Für den Linienfolger kann ein PD-Regler (Proportional-Differential) eingesetzt werden. Dieser berechnet aus der aktuellen Linienabweichung (Regelgrösse) eine Lenk-Korrektur für die Differentialgeschwindigkeit der beiden Antriebsräder.

Der P-Anteil reagiert proportional auf die aktuelle Abweichung, während der D-Anteil auf die Änderungsrate der Abweichung reagiert und Überschwinger dämpft. Ein reiner P-Regler würde auf der Linie oszillieren; der D-Anteil stabilisiert das System erheblich.

Die Reglergleichung lautet:

$$u(t) = K_p \cdot e(t) + K_d \cdot \frac{de(t)}{dt} \quad (1)$$

wobei $e(t)$ die Lageabweichung (z. B. gewichtete Sensorposition) und $u(t)$ der Lenkkorrekturwert ist. Die Parameter K_p und K_d werden empirisch durch iteratives Testen auf der Strecke eingestellt.

Eine bewährte Vorgehensweise zum Tuning ist bei Robot Research Lab dokumentiert [8].

4.2 Zustandsmaschine

Der Ablauf des Roboters (Paket suchen, aufnehmen, transportieren, abliefern) wird über eine Zustandsmaschine (Finite State Machine, FSM) gesteuert. Jeder Zustand definiert das aktuelle Verhalten des Roboters sowie die Übergangsbedingungen in den nächsten Zustand. Dadurch ist das Verhalten übersichtlich strukturiert und leicht erweiterbar [3].

5 Vergleichbare Projekte

Im Rahmen der Recherche wurden verschiedene vergleichbare Projekte aus dem Bereich autonomer Linienfolger-Roboter gesichtet. Diese dienen als Orientierung für die eigene Konzeptentwicklung.

- **Pololu 3pi+ Robot:** Ein weit verbreiteter Linienfolger-Roboter auf Arduino-Basis mit integriertem QTR-Sensorarray und PID-Regler. Er bietet eine gute Referenzimplementierung für Sensor-Kalibrierung und Reglertuning.
- **ZHAW-interne Vorprojekte:** Aus früheren PM2-Modulen existieren Erfahrungsberichte und Teilkonzepte, die als Inspirationsquelle und Fehlervermeidung dienen.
- **Open-Source Linienfolger Communities:** Auf Plattformen wie GitHub und Instructables sind zahlreiche dokumentierte Projekte mit Schaltplänen, Code und Erfahrungsberichten verfügbar, die Hinweise zu typischen Problemen (z. B. Lichtempfindlichkeit des Farbsensors, Encoder-Rauschen) geben.

Die gesichteten Projekte bestätigen die gewählten Lösungsansätze und zeigen gleichzeitig potenzielle Stolpersteine auf, die bei der weiteren Entwicklung berücksichtigt werden.

Literatur

- [1] *Barthelme LED-Sortiment – Rot, Blau, Grün, Gelb, Warmweiss, 5 mm.* Conrad Electronic. URL: <https://www.conrad.ch/de/p/barthelme-led-sortiment-rot-blau-gruen-gelb-warmweiss-rund-5-mm-120-mcd-800-mcd-2000-mcd-20000-mcd-30-35-20-1666905.html> (besucht am 06.05.2025).
- [2] *Differential Wheeled Robot.* Wikipedia. URL: https://en.wikipedia.org/wiki/Differential_wheeled_robot (besucht am 06.05.2025).
- [3] *Endlicher Automat.* Wikipedia. URL: https://de.wikipedia.org/wiki/Endlicher_Automat (besucht am 06.05.2025).
- [4] *Farbsensor TCS3200 – Beispielprojekt mit Code.* Turanis Elektronik. URL: <https://elektro.turanis.de/html/prj029/index.html> (besucht am 06.05.2025).
- [5] *Futaba Servo S-3001.* Modellsport Schweizer. URL: <https://www.modellsport.ch/RC-Elektronik/Servo/Futaba/Futaba-Servo-S-3001.html> (besucht am 06.05.2025).
- [6] *Grove Ultrasonic Ranger – Wiki.* Seeed Studio. URL: https://wiki.seeedstudio.com/Grove-Ultrasonic_Ranger/ (besucht am 06.05.2025).
- [7] *How to Build a Simple Servo-Driven Robot Arm.* Instructables. URL: <https://www.instructables.com/Servo-Driven-Robot-Arm/> (besucht am 06.05.2025).
- [8] Robot Research Lab. *PID Line Follower Tuning.* URL: <http://robotresearchlab.com/2019/02/16/pid-line-follower-tuning/> (besucht am 06.05.2025).
- [9] *Line Follower Robot – Sensor Placement and Geometry.* Society of Robots. URL: https://www.societyofrobots.com/programming_line_following.shtml (besucht am 06.05.2025).
- [10] *Omron D3V-166-2C5 – Subminiatur-Mikroschalter.* DigiKey Electronics. URL: <https://www.digikey.ch/de/products/detail/omron-electronics-inc-emc-div/D3V-166-2C5/5236792> (besucht am 06.05.2025).
- [11] *Pololu 20D Metal Gearmotors.* Pololu Corporation. URL: <https://www.pololu.com/category/167/20d-metal-gearmotors> (besucht am 06.05.2025).
- [12] *Pololu Magnetic Encoder Kit for 20D Metal Gearmotors.* Pololu Corporation. URL: <https://www.pololu.com/product/3499> (besucht am 06.05.2025).
- [13] *Reely Standard-Servo CYS-S0090 – Analog-Servo, Metallgetriebe, Stecksystem JR.* Conrad Electronic. URL: <https://www.conrad.ch/de/p/reely-standard-servo-cys-s0090-analog-servo-getriebe-material-metall-stecksystem-jr-2203091.html> (besucht am 06.05.2025).
- [14] *Robot Gripper Design – Principles and Mechanisms.* RobotShop Community. URL: <https://community.robotshop.com/blog/show/robot-gripper-design> (besucht am 06.05.2025).

- [15] *Sharp GP2Y0A41SK0F – IR-Distanzsensor 4–30 cm*. DigiKey Electronics. URL: <https://www.digikey.ch/de/products/detail/sharp-socle-technology/GP2Y0A41SK0F/3884447> (besucht am 06.05.2025).
- [16] *SparkFun Line Sensor Array – QTR-8RC*. SparkFun Electronics. URL: <https://www.sparkfun.com/products/13582> (besucht am 06.05.2025).
- [17] *TCS230/TCS3200 Farbsensor-Modul*. BerryBase. URL: <https://www.berrybase.ch/tcs230-tcs3200-farbsensor-modul> (besucht am 06.05.2025).

PES Board — main.cpp

Differential-Drive Line-Following Robot with Parcel Pickup/Drop
STM32 Nucleo F446RE · Mbed OS · PlatformIO

```

1  /**
2   * @file main.cpp
3   * @brief PES Board      differential-drive line-following robot with parcel pickup/drop.
4   *
5   * State machine:
6   *   INITIAL      FORWARD      COLOR_SCAN      HANDLE_PARCEL      INITIAL
7   *   (after 4 pickup+drop cycles: INITIAL      SLEEP)
8   *
9   * Hardware: STM32 Nucleo F446RE      Mbed OS      PlatformIO / gcc
10  */
11
12  #include "mbed.h"
13  #include "PESBoardPinMap.h"
14
15  // Drivers
16  #include "DCMotor.h"
17  #include "DebounceIn.h"
18  #include "FastPWM.h"
19  #include <Eigen/Dense>
20  #include "SensorBar.h"
21  #include "ColorSensor.h"
22  #include "Servo.h"
23  //#include "IIRFilter.h"
24
25  #define M_PIf 3.14159265358979323846f
26
27  // -----
28  // Global flags      toggled by the blue button ISR
29  // -----
30  bool do_execute_main_task = false; // true while the robot should run
31  bool do_reset_all_once    = false; // triggers a one-shot reset when task stops
32
33  // Blue button: falling edge calls the toggle function
34  DebounceIn user_button(BUTTON1);
35  void toggle_do_execute_main_fcn();
36
37  // -----
38  // Cross-detection background thread (4 ms polling)
39  // -----
40  volatile bool pickup_cross = false; // full-bar pattern seen      pickup crossing
41  volatile bool drop_cross  = false; // center-only pattern seen   drop crossing
42  volatile bool color_detected = false; // color identified this crossing; suppresses re-trigger
43  volatile int  cross_bar_count = 0;    // skips the very first bar hit so the robot gets on-track
44  volatile int  cross_lock_ticks = 0;   // cooldown counter (ticks) to avoid double-detection
45
46  SensorBar* sensor_bar_ptr = nullptr; // set before thread start; read by detect_cross_thread
47  void detect_cross_thread();
48
49  int main()
50  {
51      user_button.fall(&toggle_do_execute_main_fcn);
52
53      // -----
54      // Timing      fixed 20 ms (50 Hz) control loop
55      // -----
56      const int main_task_period_ms = 20;
57      Timer      main_task_timer;
58      main_task_timer.start();
59
60      // -----
61      // Peripherals
62      // -----

```

```

63  DigitalOut user_led(LED1);
64  DigitalOut enable_motors(PB_ENABLE_DCMOTORS); // power rail for all DC motors
65
66  // -----
67  // DC Motors
68  // M1/M2      drive wheels (motion planner disabled; direct velocity commands)
69  // M3         crane arm (motion planner enabled for smooth position control)
70  // -----
71  const float voltage_max = 12.0f;           // battery pack voltage (6.0f for single pack)
72  const float gear_ratio  = 100.00f;
73  const float kn          = 140.0f / 12.0f; // motor speed constant [rpm/V]
74
75  DCMotor motor_M1(PB_PWM_M1, PB_ENC_A_M1, PB_ENC_B_M1, gear_ratio, kn, voltage_max);
76  DCMotor motor_M2(PB_PWM_M2, PB_ENC_A_M2, PB_ENC_B_M2, gear_ratio, kn, voltage_max);
77  DCMotor motor_M3(PB_PWM_M3, PB_ENC_A_M3, PB_ENC_B_M3, gear_ratio, kn, voltage_max);
78
79  motor_M3.enableMotionPlanner();
80  motor_M3.setMaxVelocity(motor_M3.getMaxPhysicalVelocity() * 0.75f); // cap for safety
81
82  // -----
83  // Servos (pulse widths from calibration)
84  // D0      arm, D1      gripper
85  // -----
86  Servo servo_D0(PB_D0, 0.03f, 0.125f);
87  Servo servo_D1(PB_D1, 0.03f, 0.1f);
88
89  servo_D0.setMaxAcceleration(0.3f); // default 1.0e6f      slowed for smooth motion
90  servo_D1.setMaxAcceleration(0.3f);
91
92  const float servo_D0_home = 0.8f; // retracted / home pulse width
93  const float servo_D1_home = 0.6f;
94
95  // Used only in CALIBRATE_SERVO
96  float servo_input  = 0.0f;
97  int   servo_counter = 0;
98  const int loops_per_seconds = static_cast<int>(ceilf(1.0f / (0.001f *
99  ↪ static_cast<float>(main_task_period_ms))));
100
101  // -----
102  // Differential-Drive Kinematics (Eigen)
103  // Cwheel2robot maps robot-frame [v,  ]      wheel speeds.
104  // Inverted in FORWARD to get per-wheel commands from desired robot motion.
105  // -----
106  const float r_wheel      = 0.03f / 2.0f; // wheel radius [m]
107  const float b_wheel      = 0.1408f;    // wheelbase center-to-center [m]
108  const float max_speed_percentage{1.0f}; // forward speed scale factor (0 1 )
109
110  Eigen::Matrix2f Cwheel2robot;
111  Cwheel2robot <<  r_wheel / 2.0f,    r_wheel / 2.0f,
112  r_wheel / b_wheel, -r_wheel / b_wheel;
113
114  // -----
115  // Sensor Bar (8-LED optical line sensor)
116  // -----
117  const float bar_dist = 0.18f; // sensor-bar distance from wheel axis [m]
118  SensorBar  sensor_bar(PB_9, PB_8, bar_dist);
119
120  float      angle{0.0f};
121  const float default_turning_angle(1.0f); // fallback when no line detected (spin to
122  ↪ re-acquire)
123
124  // -----
125  // Line-Following PD Controller
126  // -----
127  const float Kp{12.5f};
128  const float Kd{0.25f};
129  const float dt = main_task_period_ms * 1e-3f; // control period [s]
130  float      prev_error{0.0f};
131  //IIRFilter angle_filter;
132  //angle_filter.lowPass1Init(5.0f, dt); // low-pass 5 Hz to reduce sensor noise
133
134  // Maximum tangential wheel speed [m/s], used to scale the forward velocity command
135  const float wheel_vel_max = 2.0f * M_PIf * motor_M2.getMaxPhysicalVelocity();

```

```

136 // Color Sensor
137 // -----
138 ColorSensor color_sensor(PB_3);
139
140 int current_color_run{0}; // color ID of the parcel currently being handled (0 = none)
141 bool color_done[9] = {}; // which colors have already been picked up
142 int pickups_done = 0; // completed pickup+drop cycles
143
144 // Color IDs returned by ColorSensor::getColor()
145 enum Color {
146     UNKNOWN = 0,
147     BLACK = 1,
148     WHITE = 2,
149     RED = 3,
150     YELLOW = 4,
151     GREEN = 5,
152     CYAN = 6,
153     BLUE = 7,
154     MAGENTA = 8
155 };
156
157 //int sleep_count = 0;
158
159 // -----
160 // Robot State Machine
161 // -----
162 enum RobotState {
163     INITIAL, // home servos, enable motors, choose next state
164     SLEEP, // all tasks done motors off indefinitely
165     FORWARD, // line following with PD controller
166     COLOR_SCAN, // stopped at crossing, identify parcel color
167     HANDLE_PARCEL, // execute pickup or drop sequence
168     CALIBRATE_COLOR, // debug: stream raw RGBC readings
169     CALIBRATE_SERVO // debug: sweep servo pulse width
170 } robot_state = RobotState::INITIAL;
171
172 // Hand the sensor_bar pointer to the background thread before starting it
173 sensor_bar_ptr = &sensor_bar;
174 Thread cross_thread;
175 cross_thread.start(detect_cross_thread);
176
177 bool is_pickup = true; // true = next HANDLE_PARCEL is a pickup, false = drop
178
179 // -----
180 // Crane Parameters per color, per operation
181 // m3_offset : signed rotation added to M3 current position (extend/retract)
182 // srv_out_d0 : arm servo pulse width during operation
183 // srv_out_d1 : gripper servo pulse width during operation
184 // -----
185 struct ParcelParams { float m3_offset, srv_out_d0, srv_out_d1; };
186
187 const ParcelParams pickup_params[9] = {
188     {}, // UNKNOWN
189     {}, // BLACK
190     {}, // WHITE
191     { 1.75f, 0.95f, 0.15f }, // RED down right
192     { 1.75f, 0.95f, 0.2f }, // YELLOW up right
193     { -0.55f, 0.85f, 0.1f }, // GREEN down left
194     {}, // CYAN
195     { -0.55f, 1.0f, 0.15f }, // BLUE up left
196     {}, // MAGENTA
197 };
198
199 const ParcelParams drop_params[9] = {
200     {}, // UNKNOWN
201     {}, // BLACK
202     {}, // WHITE
203     { 1.75f, 0.95f, 0.1f }, // RED
204     { 1.75f, 0.9f, 0.2f }, // YELLOW
205     { -0.55f, 0.85f, 0.1f }, // GREEN
206     {}, // CYAN
207     { -0.6f, 1.1f, 0.17f }, // BLUE
208     {}, // MAGENTA
209 };
210

```

```

211 // =====
212 // Main loop      executes every main_task_period_ms milliseconds
213 // =====
214 while (true) {
215     main_task_timer.reset();
216
217     if (do_execute_main_task) {
218         // -----
219         // Active: blue button has been pressed      run the state machine
220         // -----
221         switch (robot_state) {
222
223             // -----
224             // INITIAL: enable motors, home servos, then start driving.
225             // Waits until both servos reach home position before advancing.
226             // -----
227             case RobotState::INITIAL: {
228                 enable_motors    = 1;
229                 pickup_cross     = false;
230                 drop_cross       = false;
231                 cross_bar_count  = 0;
232
233                 if (!servo_D0.isEnabled()) servo_D0.enable();
234                 if (!servo_D1.isEnabled()) servo_D1.enable();
235                 servo_D0.setPulseWidth(servo_D0_home);
236                 servo_D1.setPulseWidth(servo_D1_home);
237
238                 if (fabsf(servo_D0.getPulseWidth() - servo_D0_home) < 0.02f &&
239                     fabsf(servo_D1.getPulseWidth() - servo_D1_home) < 0.02f) {
240                     robot_state = (pickups_done >= 4) ? RobotState::SLEEP :
241                                     ↳ RobotState::FORWARD;
242                 }
243                 break;
244             }
245
246             // -----
247             // SLEEP: mission complete      keep motors off indefinitely
248             // -----
249             case RobotState::SLEEP: {
250                 enable_motors = 0;
251                 break;
252             }
253
254             // -----
255             // FORWARD: follow the black line using PD control on sensor angle
256             // -----
257             case RobotState::FORWARD: {
258                 if (sensor_bar.isAnyLedActive()) {
259                     uint8_t raw = sensor_bar.getRaw();
260
261                     // Count active LEDs to detect a wide crossing ( 4 LEDs lit)
262                     int active_leds = 0;
263                     for (int i = 0; i < 8; i++) {
264                         if (raw & (1 << i)) active_leds++;
265                     }
266
267                     // During a drop run a wide bar means we're at the crossing;
268                     // bias angle slightly left to align with the drop position.
269                     if (is_pickup == false && active_leds >= 4) {
270                         angle = -0.15f;
271                     } else {
272                         angle = sensor_bar.getAvgAngleRad();
273                     }
274
275                     // Cross detected by background thread      stop and scan color
276                     if ((pickup_cross || drop_cross) && !color_detected) {
277                         prev_error = 0.0f;
278                         robot_state = RobotState::COLOR_SCAN;
279                         break;
280                     }
281                 } else {
282                     //angle = 0.0f; // straight ahead when no line
283                     angle = default_turning_angle; // no line      spin to re-acquire
284                 }

```

```

285 // PD controller: error = measured angle, output = angular velocity
286 color_detected = false;
287 const float angle_threshold = -0.3;
288 const float derivative_threshold = Kd * angle_threshold; // Kd * -0.3
289 const float derivative_correction_factor = 16.0f;
290 float derivative = (angle - prev_error) / dt;
291 float omega = -(Kp * angle + Kd * derivative);
292 prev_error = angle;
293
294 //if (Kd * derivative < derivative_threshold) {
295 //    omega *= derivative_correction_factor;
296 //    //printf("omega: %f, angle: %f, Kd, %f, derivative %f\n", omega,
297 //        ↪ angle, Kd, derivative);
298 //}
299
300 // slow down on sharp turns, speed up on straights
301 //float speed_factor = 1.0f - fabsf(angle) / default_turning_angle;
302 //speed_factor = (speed_factor < 0.3f) ? 0.3f : speed_factor;
303
304 // Convert desired robot velocities to individual wheel speeds [rot/s]
305 Eigen::Vector2f robot_coord = {max_speed_percentage * wheel_vel_max *
306     ↪ r_wheel, omega};
307 //Eigen::Vector2f robot_coord = {speed_factor * max_speed_percentage *
308     ↪ wheel_vel_max * r_wheel, omega};
309 Eigen::Vector2f wheel_speed = Cwheel2robot.inverse() * robot_coord;
310
311 // M2 is mounted mirrored on the chassis      negate its velocity
312 motor_M1.setVelocity( wheel_speed(0) / (2.0f * M_PIf));
313 motor_M2.setVelocity(-wheel_speed(1) / (2.0f * M_PIf));
314
315 //printf("angle: %f | omega: %f\n | lin_speed: %f\n", angle, omega,
316     ↪ max_speed_percentage * wheel_vel_max * r_wheel);
317 break;
318 }
319
320 // -----
321 // COLOR_SCAN: robot stopped at crossing      identify parcel color.
322 // Decides whether to pickup, drop, or ignore this crossing.
323 // -----
324 case RobotState::COLOR_SCAN: {
325     motor_M1.setVelocity(0.0f);
326     motor_M2.setVelocity(0.0f);
327     color_sensor.switchLed(ON);
328
329     int detected = color_sensor.getColor();
330
331     if (detected == YELLOW || detected == GREEN ||
332         detected == RED || detected == BLUE) {
333         color_sensor.switchLed(OFF);
334
335         if (!color_done[detected] && pickup_cross && current_color_run == 0) {
336             // New color at a pickup cross      begin pickup sequence
337             color_done[detected] = true;
338             current_color_run = detected;
339             color_detected = true;
340             //printf("Color: %s\n",
341                 ↪ ColorSensor::getColorString(current_color_run));
342             is_pickup = true;
343             robot_state = RobotState::HANDLE_PARCEL;
344             //robot_state = RobotState::FORWARD;
345         } else if (current_color_run == detected) {
346             // Same color as carried parcel      this is the drop-off spot
347             drop_cross = false;
348             pickup_cross = false;
349             is_pickup = false;
350             robot_state = RobotState::HANDLE_PARCEL;
351             //robot_state = RobotState::FORWARD;
352         } else {
353             // Wrong color or already handled      skip and keep driving
354             drop_cross = false;
355             pickup_cross = false;
356             robot_state = RobotState::FORWARD;
357         }
358     }
359 }
360 break;

```

```

355     }
356
357     // -----
358     // HANDLE_PARCEL: sequenced crane movement for pickup or drop.
359     // Each step polls a tolerance condition before advancing to the next.
360     //
361     // 1. MOVE_OUT_SRV1      position gripper servo (D1)
362     // 2. MOVE_OUT_SRV0      position arm servo (D0)
363     // 3. MOVE_OUT_M3        extend crane (M3)
364     // 4. MOVE_BACK_M3       retract crane
365     // 5. MOVE_BACK_SRVS     return both servos to home
366     // -----
367     case RobotState::HANDLE_PARCEL: {
368         motor_M1.setVelocity(0.0f);
369         motor_M2.setVelocity(0.0f);
370         user_led = 1;
371
372         static enum class HandleStep {
373             MOVE_OUT_SRV1,
374             MOVE_OUT_SRV0,
375             MOVE_OUT_M3,
376             MOVE_BACK_M3,
377             MOVE_BACK_SRVS,
378         } step = HandleStep::MOVE_OUT_SRV1;
379         static float m3_target = 0.0f;
380
381         // Select crane parameters for the current color and operation
382         const ParcelParams& p = is_pickup ? pickup_params[current_color_run]
383                                         : drop_params[current_color_run];
384
385         switch (step) {
386             case HandleStep::MOVE_OUT_SRV1: {
387                 if (!servo_D1.isEnabled()) servo_D1.enable();
388                 servo_D1.setPulseWidth(p.srv_out_d1);
389                 if (fabsf(servo_D1.getPulseWidth() - p.srv_out_d1) < 0.02f)
390                     step = HandleStep::MOVE_OUT_SRV0;
391                 break;
392             }
393             case HandleStep::MOVE_OUT_SRV0: {
394                 if (!servo_D0.isEnabled()) servo_D0.enable();
395                 servo_D0.setPulseWidth(p.srv_out_d0);
396                 if (fabsf(servo_D0.getPulseWidth() - p.srv_out_d0) < 0.02f) {
397                     m3_target = motor_M3.getRotation() + p.m3_offset;
398                     step = HandleStep::MOVE_OUT_M3;
399                 }
400                 break;
401             }
402             case HandleStep::MOVE_OUT_M3: {
403                 motor_M3.setRotation(m3_target);
404                 if (fabsf(motor_M3.getRotation() - m3_target) < 0.05f) {
405                     m3_target = motor_M3.getRotation() - p.m3_offset;
406                     step = HandleStep::MOVE_BACK_M3;
407                 }
408                 break;
409             }
410             case HandleStep::MOVE_BACK_M3: {
411                 motor_M3.setRotation(m3_target);
412                 if (fabsf(motor_M3.getRotation() - m3_target) < 0.05f)
413                     step = HandleStep::MOVE_BACK_SRVS;
414                 break;
415             }
416             case HandleStep::MOVE_BACK_SRVS: {
417                 servo_D0.setPulseWidth(servo_D0_home);
418                 servo_D1.setPulseWidth(servo_D1_home);
419                 if (fabsf(servo_D0.getPulseWidth() - servo_D0_home) < 0.02f &&
420                     fabsf(servo_D1.getPulseWidth() - servo_D1_home) < 0.02f) {
421                     step = HandleStep::MOVE_OUT_SRV1;
422                     pickup_cross = false;
423                     drop_cross = false;
424
425                     // Lock out cross detection for 125 ticks (    0.5 s at 4 ms
426                     //    ↳ polling)
427                     // to prevent immediately re-triggering on the same crossing.
428                     cross_lock_ticks = 125;
429                     pickups_done++;

```

```

429         if (!is_pickup) current_color_run = 0; // clear carried color
430             ↳ after drop
431         robot_state = RobotState::INITIAL;
432     }
433     break;
434 }
435 break;
436 }
437
438 // -----
439 // CALIBRATE_COLOR: stream raw RGBC values for sensor tuning
440 // -----
441 case RobotState::CALIBRATE_COLOR: {
442     color_sensor.switchLed(ON);
443     const float* colors = color_sensor.readColor();
444     printf("R: %.2f\t G: %.2f\t B: %.2f\t C: %.2f\n",
445           colors[0], colors[1], colors[2], colors[3]);
446     break;
447 }
448
449 // -----
450 // CALIBRATE_SERVO: slowly sweep D1 pulse width and print it
451 // -----
452 case RobotState::CALIBRATE_SERVO: {
453     if (!servo_D0.isEnabled()) servo_D0.enable();
454     if (!servo_D1.isEnabled()) servo_D1.enable();
455
456     //servo_D0.setPulseWidth(servo_input);
457     servo_D1.setPulseWidth(servo_input);
458
459     // Increment pulse width by 0.005 once per second
460     if ((servo_input < 1.0f) &&
461         (servo_counter % loops_per_seconds == 0) &&
462         (servo_counter != 0))
463         servo_input += 0.005f;
464     servo_counter++;
465
466     printf("Pulse width: %f\n", servo_input);
467     break;
468 }
469
470 default:
471     break;
472 }
473
474 } else {
475     // -----
476     // Inactive: blue button not pressed.
477     // The reset block runs exactly once when execution is stopped.
478     // -----
479     if (do_reset_all_once) {
480         do_reset_all_once = false;
481
482         // Disable servos and reset calibration state
483         servo_D0.disable();
484         servo_D1.disable();
485         servo_input = 0.0f;
486         servo_counter = 0;
487     }
488 }
489
490 // toggling the user led
491 //user_led = !user_led;
492
493 // Sleep for the remainder of the 20 ms period (non-blocking)
494 int main_task_elapsed_time_ms =
495     ↳ duration_cast<milliseconds>(main_task_timer.elapsed_time()).count();
496 if (main_task_period_ms - main_task_elapsed_time_ms < 0)
497     printf("Warning: Main task took longer than main_task_period_ms\n");
498 else
499     thread_sleep_for(main_task_period_ms - main_task_elapsed_time_ms);
500 }
501

```



```

502 // -----
503 // Button ISR      toggles task execution on falling edge (button press)
504 // -----
505 void toggle_do_execute_main_fcn()
506 {
507     do_execute_main_task = !do_execute_main_task;
508     if (do_execute_main_task)
509         do_reset_all_once = true; // schedule one-shot reset on re-enable
510 }
511
512 // -----
513 // Background thread      polls the sensor bar every 4 ms for crossing detection.
514 // -----
515 // Pattern recognition:
516 //   raw == 0xff  (all 8 LEDs active)      full bar      sets pickup_cross (rising edge only)
517 //   raw == 0x3c  (center 4 LEDs active)  center        sets drop_cross   (rising edge only)
518 // -----
519 // cross_lock_ticks: cooldown after HANDLE_PARCEL to prevent double-triggering.
520 // cross_bar_count:  skips the first full-bar hit so the robot can get onto the track.
521 // -----
522 void detect_cross_thread()
523 {
524     bool prev_full_line = false;
525     bool prev_cross_line = false;
526
527     while (true) {
528         uint8_t raw = sensor_bar_ptr->getRow();
529         bool full_line = (raw == 0xff);
530         bool cross_line = (raw == 0x3c);
531
532         if (cross_lock_ticks > 0) cross_lock_ticks--;
533
534         if (cross_lock_ticks == 0) {
535             // Rising edge on center-only pattern      drop crossing
536             if (!drop_cross && cross_line && !prev_cross_line)
537                 drop_cross = true;
538
539             // Rising edge on full bar      pickup crossing (skip the very first hit)
540             if (!pickup_cross && full_line && !prev_full_line) {
541                 // cross_bar_count skips the first cross to get on the track
542                 if (cross_bar_count == 0) cross_bar_count++;
543                 else pickup_cross = true;
544             }
545         }
546
547         prev_full_line = full_line;
548         prev_cross_line = cross_line;
549         ThisThread::sleep_for(4ms);
550     }
551 }

```